

**PANEVROPSKI UNIVERZITET APEIRON, BANJA LUKA
FAKULTET INFORMACIONIH TEHNOLOGIJA**

Redovne studije

Smjer : “Inženjering Informacionih Tehnologija“

Predmet:

RDBMS (SQL Administracija i CASE Alati)

**SISTEM ZA PRAĆENJE SPORTSKIH
TAKMIČENJA
(Seminarski rad)**

**Predmetni nastavnik
Doc.dr Dražen Marinković**

**Asistent
Pero Ranilović**

**Student: Ivan Pavlović
Br. Indeksa: 92-20-RITP/S**

Banja Luka, avgust 2026

SADRŽAJ

UVOD	1
1. Analiza potreba i ciljeva sistema	2
2. Kreiranje baze podataka.....	3
2.1 Kreiranje tabela	4
2.1.1. Tabela Drzave	4
2.1.2. Tabela Gradovi.....	4
2.1.3. Tabela Sportovi.....	5
2.1.4. Tabela Organizatori	6
2.1.5. Tabela Sezone	6
2.1.6. Tabela Takmicenja.....	7
2.1.7. Tabela Ekipe	8
2.1.8. Tabela Sportisti	8
2.1.9. Tabela Utakmice	9
2.1.10. Tabela Ulaznice	11
2.1.11. Tabela Placanja	12
2.1.12. Tabela za praćenje promjena utakmica.....	12
2.2. USPOSTAVLJANJE INTEGRITETA NAD PODACIMA	13
2.2.1. Integritet na nivou entiteta	13
2.2.2. Integritet na nivou domena	14
2.2.3. Ograničenja za tabelu Utakmice	14
2.2.4. Ograničenja za tabelu Placanja	14
2.2.5. Referencijalni integritet u bazi.....	15
2.3. Kreiranje indeksa	15
2.3.1. Indeks na kolonama DrzavaID i Naziv u tabeli Gradovi	16
2.3.2. Indeks na kolonama SportID i SezonaID u tabeli Takmicenja.....	16
2.3.3. Indeks na kolonama Prezime i Ime u tabeli Sportisti	16
2.3.4. Indeks na kolonama EkipaID i DatumDo u tabeli ClanoviEkipa	16
2.3.5. Indeks na koloni DatumVrijeme u tabeli Utakmice	17
2.3.6. Kompozitni indeks na kolonama TakmicenjeID i StatusUtakmice u tabeli Utakmice	17
2.3.7. Indeks na koloni SportistaID u tabeli Nastupi	17
2.3.8. Kompozitni indeks na kolonama StatusUlaznice i UtakmicaID u tabeli Ulaznice	17
2.3.9. Indeks na koloni DatumPlacanja u tabeli Placanja	18

2.3.10. Filtrirani jedinstveni indeks na koloni BrojTransakcije u tabeli Placanja	18
2.4. Unos podataka u tabele	18
2.4.1. Unos podataka u tabelu Drzave	18
2.4.2. Unos podataka u tabelu Gradovi.....	19
2.4.3. Unos podataka u tabelu Sportovi	19
2.4.4. Unos podataka u tabelu Organizatori.....	19
2.4.5. Unos podataka u tabelu Sezone	19
2.4.6. Unos podataka u tabelu Takmicenja	20
2.4.7. Unos podataka u tabelu Faze	20
2.4.8. Unos podataka u tabelu Ekipe.....	20
2.4.9. Unos podataka u tabelu Sportisti	21
2.4.10. Unos podataka u tabelu ClanoviEkipa.....	21
2.4.11. Unos podataka u tabelu Objekti.....	21
2.4.12. Unos podataka u tabelu Sudije.....	22
2.4.13. Unos podataka u tabelu Utakmice	22
2.4.14. Unos podataka u tabelu Nastupi	22
2.4.15. Unos podataka u tabelu Kazne.....	23
2.4.16. Unos podataka u tabelu Sponzori	23
2.4.17. Unos podataka u tabelu TakmicenjeSponzori.....	23
2.4.18. Unos podataka u tabelu Ulaznice.....	24
2.4.19. Unos podataka u tabelu Placanja	24
2.5. Kreiranje pogleda.....	24
2.5.1. Pogled vw_RasporedUtakmica.....	25
2.5.2. Pogled vw_ProdajaPoUtakmici	26
2.5.3. Pogled vw_StatistikaSportista	26
2.6. Kreiranje korisničkih funkcija	27
2.6.1. Funkcija fn_Starost	27
2.6.2. Funkcija fn_PrihodUtakmice	28
2.6.3. Funkcija fn_UtakmiceEkipe	28
2.7. Kreiranje uskladištenih procedura radi automatizacije.....	29
2.7.1. Procedura sp_EvidentirajRezultat.....	29
2.7.2. Procedura sp_ProdajUlaznicu	31
2.7.3. Procedura sp_RasporedZaPeriod	32
2.7.4. Procedura sp_PrihodPoTakmicenju.....	33

2.8. KREIRANJE TRIGERA ZA AUTOMATSKO IZVRŠAVANJE RADNJI.....	33
2.8.1. Trigger trg_Utakmice_Dnevnik.....	34
2.8.2. Trigger trg_Ulaznice_CijenaDnevnik.....	35
2.8.3. Trigger trg_Utakmice_Kapacitet	35
2.9. Primjeri DDL i DML naredbi u bazi podataka	36
2.9.1. ALTER naredba za izmjenu strukture	36
2.9.2. INSERT naredba za unos podataka	36
2.9.3. UPDATE naredba za izmjenu podataka	37
2.9.4. DELETE naredba za brisanje podataka	37
2.9.5. TRUNCATE naredba za brzo brisanje svih podataka	37
2.10. Primjeri korištenja različitih funkcija u bazi podataka	37
2.10.1. Datumske funkcije	38
2.10.2. String funkcije.....	38
2.10.3. Agregatne funkcije.....	38
2.10.4. Numeričke funkcije.....	39
2.11. PRIMJERI JOIN UPITA ZA KOMBINOVANJE PODATAKA IZ VIŠE TABELA	39
2.11.1. INNER JOIN.....	39
2.11.2. LEFT JOIN	39
2.11.3. RIGHT JOIN.....	40
2.11.4. FULL OUTER JOIN.....	40
2.11.5. CROSS JOIN	40
2.11.6. SELF JOIN.....	41
2.12. Primjeri podupita i skupovnih operatora.....	41
2.12.1. Podupiti	41
2.12.2. Skupovni operatori.....	42
2.13. Korisnici, bezbjednost i uloge.....	43
2.13.1. Kreiranje uloga.....	43
2.13.2. Dodjela prava ulogama	43
2.13.3. Kreiranje korisnika.....	44
2.13.4. Testiranje sigurnosnog konteksta.....	44
2.14. ETL PROCESI.....	44
2.14.1. Tabela StageSportisti	45
2.14.2. Procedura sp_UveziSportiste	45
2.14.3. Tabela DnevniPrihod	46

2.14.4. Procedura sp_UcitajDnevniPrihod.....	46
2.15. Kreiranje rezervne kopije baze podataka	47
2.15.1. Kreiranje potpunog backup-a.....	47
2.15.2. Diferencijalni backup.....	48
2.15.3. Backup loga transakcija	48
2.15.4. Copy-only backup	49
2.15.5. Ostale vrste backup-a.....	49
2.15.6. Informacije o backup-u	50
2.15.7. Restauracija baze iz backup-a	50
2.16. Publikacija i transakcijska replikacija.....	51
2.16.1. Konfiguracija Distributora	51
2.16.2. Kreiranje publikacije.....	51
2.16.3. Dodavanje artikala	52
2.16.4. Kreiranje pretplate	52
2.17. Plan testiranja.....	53
2.17.1. Test integriteta entiteta.....	53
2.17.2. Test domenskog integriteta	53
2.17.3. Test referencijalnog integriteta	54
2.17.4. Test trigera za dnevnik utakmica	54
2.17.5. Test trigera za kapacitet objekta	54
2.17.6. Test procedura.....	55
2.17.7. Test ETL procesa	55
2.17.8. Test sigurnosti.....	56
2.18.9. Test transakcije	56
2.18.10. Test indeksa	57
2.19. ER DIJAGRAM I OPIS RELACIJA	57
2.19.1. Entiteti i njihovi atributi.....	57
2.19.2. Veze između entiteta.....	58
2.19.3. Grafički prikaz ER dijagrama	60
ZAKLJUČAK.....	61
POPIS SLIKA.....	62

UVOD

Savremeni sportski događaji generišu veliku količinu podataka koji se odnose na takmičenja, ekipe, sportiste, rezultate, prodaju ulaznica i finansijske transakcije. Organizacije koje upravljaju sportskim takmičenjima suočavaju se sa potrebom za efikasnim upravljanjem ovim informacijama kako bi osigurale pravovremeno izvještavanje, tačnost podataka i zadovoljstvo svih učesnika. Tradicionalni načini vođenja evidencije, poput papirne dokumentacije ili pojednostavljenih tabelarnih prikaza, više nisu dovoljni da odgovore na zahtjeve brzine, tačnosti i pouzdanosti koji se danas podrazumijevaju.

Digitalizacija poslovnih procesa u sportu postaje neizostavan dio modernog upravljanja. Informacioni sistemi zasnovani na relacionim bazama podataka omogućavaju automatizaciju ključnih aktivnosti, smanjenje mogućnosti grešaka i unapređenje korisničkog iskustva. Sistem za praćenje sportskih takmičenja predstavlja važan segment takvog pristupa, jer omogućava centralizovano upravljanje podacima o takmičenjima, ekipama, sportistima, utakmicama i prodaji ulaznica.

Projektovanje i implementacija baze podataka predstavljaju osnovu svakog pouzdanog informacionog sistema. Kvalitetno dizajniran relacioni model omogućava dosljedno čuvanje podataka, njihovu povezanost i jednostavnu obradu. U kontekstu sportskih takmičenja, takav model obuhvata evidenciju sportova, sezona, organizatora, takmičenja, ekipa, sportista, utakmica, nastupa, ulaznica i plaćanja, uz jasno definisane veze između tih elemenata.

Za realizaciju baze podataka korišten je Microsoft SQL Server sa nivoom kompatibilnosti 100, što odgovara SQL Server 2008 okruženju. Posebna pažnja posvećena je očuvanju integriteta podataka, kao i optimizaciji performansi kroz pravilno definisane indekse. Pored osnovnih tabela, korišćeni su i napredni objekti baze podataka, uključujući pogleda, uskladištene procedure, trigere i korisnički definisane funkcije, čime se omogućava efikasnija obrada podataka i automatizacija poslovnih procesa. Također, implementirani su ETL procesi, sigurnosni mehanizmi kroz korisničke uloge, te primjeri backup-a i transakcijske replikacije.

1. Analiza potreba i ciljeva sistema

Prije početka projektovanja baze podataka, neophodno je izvršiti detaljnu analizu zahtjeva koja definiše šta sistem treba da podržava. U slučaju sistema za praćenje sportskih takmičenja, osnovni zahtjevi obuhvataju evidentiranje svih sportskih disciplina koje se prate, sa pripadajućim karakteristikama kao što su tip sporta, minimalan i maksimalan broj igrača. Svaki sport može biti ekipni ili individualni, što utiče na način organizacije takmičenja.

Sistem mora omogućiti evidenciju geografskih podataka, uključujući države i gradove, što je važno za praćenje lokacija ekipa, sportista i sportskih objekata. Organizatori takmičenja se evidentiraju sa svojim osnovnim podacima, dok se sezone definišu sa datumima početka i završetka, što omogućava praćenje takmičenja u vremenskom kontekstu. Takmičenja su centralni entitet sistema jer povezuju sportove, organizatore i sezone. Svako takmičenje ima svoj naziv, rang, datume početka i završetka, status i nagradni fond. Rang takmičenja može biti lokalno, državno, regionalno ili međunarodno, što omogućava različite nivoe organizacije. Unutar takmičenja definišu se faze, kao što su ligaški dio ili završnica, što omogućava praćenje napredovanja kroz različite etape. Ekipe se evidentiraju sa svojim nazivom, skraćenim nazivom, gradom, sportom, budžetom i datumom osnivanja. Sportisti su povezani sa državama i mogu biti članovi ekipa kroz tabelu koja prati njihovo članstvo kroz vrijeme. Utakmice se održavaju na sportskim objektima, a za svaku utakmicu evidentiraju se domaća i gostujuća ekipa, sudija, datum i vrijeme, status, rezultat i broj gledalaca.

Nastupi sportista na utakmicama omogućavaju praćenje individualne statistike, uključujući broj poena, asistencija i minuta nastupa. Kazne se evidentiraju za sportiste ili ekipe, sa vrstom kazne, minutom i novčanim iznosom. Prodajni dio sistema obuhvata sponzore, ulaznice i plaćanja, što omogućava praćenje finansijskih aspekata takmičenja.

Izveštaji koji su potrebni obuhvataju pregled rasporeda utakmica, prodaju ulaznica po utakmicama, statistiku sportista, te dnevni prihod po takmičenjima. Ovi izveštaji su ključni za operativno upravljanje i strateško planiranje.

2. Kreiranje baze podataka

Prvi korak u izradi projekta jeste kreiranje baze podataka pod nazivom SportskaTakmicenja. Time se uspostavlja osnovna struktura sistema u kojoj će se čuvati i organizovati svi podaci vezani za praćenje sportskih takmičenja. U okviru sistema SQL Server, baza podataka predstavlja logičku cjelinu koja objedinjuje sve tabele, veze i ostale objekte potrebne za rad aplikacije.

Nakon kreiranja baze podataka, aktivira se za dalji rad korištenjem naredbe USE, kojom se definiše da će se sve naredne operacije izvršavati nad bazom SportskaTakmicenja. Aktivna baza može se izabrati i putem grafičkog korisničkog interfejsa u okviru SQL Server Management Studio okruženja.

```
CREATE DATABASE SportskaTakmicenja;  
GO  
USE SportskaTakmicenja;  
GO
```

Slika 1 - Kreiranje i aktiviranje baze podataka

Organizacija objekata unutar baze podataka postignuta je korištenjem shema. Sheme predstavljaju logičke kontejnere koji omogućavaju grupisanje objekata prema njihovoj namjeni. U okviru ove baze definisano je nekoliko shema. Shema sif sadrži šifarnike, odnosno tabele sa osnovnim podacima poput država, gradova i sportova. Shema sport obuhvata tabele vezane za sportsko poslovanje, uključujući organizatore, sezone, takmičenja, faze, ekipe, sportiste, članove ekipa, objekte, sudije, utakmice, nastupe i kazne.

```
CREATE SCHEMA sif AUTHORIZATION dbo;  
GO  
CREATE SCHEMA sport AUTHORIZATION dbo;  
GO  
CREATE SCHEMA prodaja AUTHORIZATION dbo;  
GO  
CREATE SCHEMA izvjestaji AUTHORIZATION dbo;  
GO  
CREATE SCHEMA audit AUTHORIZATION dbo;  
GO  
CREATE SCHEMA etl AUTHORIZATION dbo;  
GO
```

Slika 2 - Kreiranje shema u bazi podataka

Ovakva podjela poboljšava preglednost baze podataka i olakšava upravljanje objektima. Također, omogućava precizniju dodjelu prava pristupa na nivou poslovnih cjelina, što je važno za bezbjednost sistema. Na primjer, korisnicima koji rade na sportskom dijelu može se dati pristup samo shemi sport, dok analitičari mogu imati pristup shemi izvjestaji.

2.1 Kreiranje tabela

Nakon kreiranja baze podataka i shema, slijedi definisanje tabela koje će čuvati podatke o glavnim entitetima sistema. Svaka tabela sadrži kolone sa odgovarajućim tipovima podataka, a za svaku je definisan primarni ključ radi jedinstvene identifikacije zapisa. Veze između tabela ostvarene su pomoću stranih ključeva.

2.1.1. Tabela Drzave

Tabela Drzave predstavlja šifarnik država. Ova tabela je ključna za geografsku kategorizaciju i omogućava povezivanje gradova, sportista i sudija sa odgovarajućim državama. Prva kolona u ovoj tabeli je DrzavaID, koja je definisana kao IDENTITY kolona. Identity svojstvo znači da će SQL Server automatski generisati jedinstvenu numeričku vrijednost za svaki novi red, počevši od 1 i povećavajući se za 1. Ova kolona je postavljena kao primarni ključ tabele.

DrzavaID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Drzave PRIMARY KEY

Kolona Naziv je definisana kao NVARCHAR(80) sa ograničenjem NOT NULL i UNIQUE, što znači da svaka država mora imati naziv i da naziv mora biti jedinstven. NVARCHAR tip podataka omogućava čuvanje Unicode znakova, što je važno za podršku različitih jezika i specijalnih karaktera. Kolona ISOKod je također obavezna i jedinstvena, definisana kao NCHAR(3) što odgovara standardnim trocifrenim ISO kodovima država.

Naziv NVARCHAR(80) NOT NULL CONSTRAINT UQ_Drzave_Naziv UNIQUE,
ISOKod NCHAR(3) NOT NULL CONSTRAINT UQ_Drzave_ISO UNIQUE

2.1.2. Tabela Gradovi

Tabela Gradovi definiše gradove koji su povezani sa državama. GradID je primarni ključ sa identity svojstvom. DrzavaID je strani ključ koji povezuje grad sa odgovarajućom državom. Naziv

grada je obavezan, dok su PostanskiBroj i BrojStanovnika opciona polja. Ograničenje UQ_Gradovi osigurava da kombinacija države i naziva grada bude jedinstvena.

```
GradID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Gradovi PRIMARY KEY,  
DrzavaID INT NOT NULL,  
Naziv NVARCHAR(80) NOT NULL,  
PostanskiBroj NVARCHAR(15) NULL,  
BrojStanovnika INT NULL,  
CONSTRAINT FK_Gradovi_Drzave FOREIGN KEY(DrzavaID) REFERENCES  
sif.Drzave(DrzavaID),  
CONSTRAINT UQ_Gradovi UNIQUE(DrzavaID,Naziv),  
CONSTRAINT CHK_Gradovi_Stanoavnici CHECK(BrojStanovnika IS NULL OR  
BrojStanovnika>=0)
```

2.1.3. Tabela Sportovi

Tabela Sportovi predstavlja šifarnik sportskih disciplina. SportID je primarni ključ sa identity svojstvom. Naziv sporta je obavezan i jedinstven. TipSporta definiše da li je sport ekipni ili individualni, sa CHECK ograničenjem koje dozvoljava samo ove dvije vrijednosti. MinimalanBrojIgraca i MaksimalanBrojIgraca definišu dozvoljeni raspon igrača za dati sport.

```
SportID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Sportovi PRIMARY KEY,  
Naziv NVARCHAR(60) NOT NULL CONSTRAINT UQ_Sportovi_Naziv UNIQUE,  
TipSporta NVARCHAR(15) NOT NULL,  
MinimalanBrojIgraca INT NOT NULL,  
MaksimalanBrojIgraca INT NOT NULL,  
CONSTRAINT CHK_Sportovi_Tip CHECK(TipSporta IN(N'Ekipni',N'Individualni')),  
CONSTRAINT CHK_Sportovi_Igraci CHECK(MinimalanBrojIgraca>0 AND  
MaksimalanBrojIgraca>=MinimalanBrojIgraca)
```

2.1.4. Tabela Organizatori

Tabela Organizatori čuva podatke o organizatorima sportskih takmičenja. OrganizatorID je primarni ključ sa identity svojstvom. Naziv organizatora je obavezan i jedinstven, a JIB je također jedinstven. Email i Telefon su opciona polja, dok DatumOsnivanja omogućava praćenje starosti organizacije. Aktivan je logičko polje sa podrazumijevanom vrijednošću 1.

```
OrganizatorID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Organizatori PRIMARY KEY,  
Naziv NVARCHAR(120) NOT NULL CONSTRAINT UQ_Organizatori_Naziv UNIQUE,  
JIB NVARCHAR(20) NOT NULL CONSTRAINT UQ_Organizatori_JIB UNIQUE,  
Email NVARCHAR(100) NULL,  
Telefon NVARCHAR(25) NULL,  
DatumOsnivanja DATE NULL,  
Aktivan BIT NOT NULL CONSTRAINT DF_Organizatori_Aktivan DEFAULT 1,  
CONSTRAINT CHK_Organizatori_Email CHECK(Email IS NULL OR Email LIKE N'%@%._%')
```

2.1.5. Tabela Sezone

Tabela Sezone definiše sezone u kojima se održavaju takmičenja. SezonaID je primarni ključ sa identity svojstvom. Naziv sezone je obavezan i jedinstven. DatumPocetka i DatumZavrsetka su obavezna polja sa CHECK ograničenjem koje osigurava da je datum završetka veći od datuma početka. Aktivna označava da li je sezona trenutno aktivna.

```
SezonaID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Sezone PRIMARY KEY,  
Naziv NVARCHAR(30) NOT NULL CONSTRAINT UQ_Sezone_Naziv UNIQUE,  
DatumPocetka DATE NOT NULL,  
DatumZavrsetka DATE NOT NULL,  
Aktivna BIT NOT NULL CONSTRAINT DF_Sezone_Aktivna DEFAULT 0,  
CONSTRAINT CHK_Sezone_Datumi CHECK(DatumZavrsetka>DatumPocetka)
```

2.1.6. Tabela Takmicenja

Tabela Takmicenja je centralni entitet sistema koji povezuje sportove, organizatore i sezone. TakmicenjeID je primarni ključ sa identity svojstvom. SportID, OrganizatorID i SezonaID su strani ključevi. Naziv takmičenja je obavezan, a kombinacija sa SezonaID je jedinstvena. RangTakmicenja može imati vrijednosti Lokalno, Državno, Regionalno ili Međunarodno. StatusTakmicenja može biti Planirano, U toku, Završeno ili Otkazano. NagradniFond je decimalno polje sa podrazumijevanom vrijednošću 0.

```
TakmicenjeID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Takmicenja PRIMARY
KEY,
SportID INT NOT NULL,
OrganizatorID INT NOT NULL,
SezonaID INT NOT NULL,
Naziv NVARCHAR(120) NOT NULL,
RangTakmicenja NVARCHAR(20) NOT NULL,
DatumPocetka DATE NOT NULL,
DatumZavrsetka DATE NOT NULL,
StatusTakmicenja NVARCHAR(20) NOT NULL CONSTRAINT DF_Takmicenja_Status
DEFAULT N'Planirano',
NagradniFond DECIMAL(14,2) NOT NULL CONSTRAINT DF_Takmicenja_Fond DEFAULT
0,
CONSTRAINT FK_Takmicenja_Sport FOREIGN KEY(SportID) REFERENCES
sif.Sportovi(SportID),
CONSTRAINT FK_Takmicenja_Organizator FOREIGN KEY(OrganizatorID) REFERENCES
sport.Organizatori(OrganizatorID),
CONSTRAINT FK_Takmicenja_Sezona FOREIGN KEY(SezonaID) REFERENCES
sport.Sezone(SezonaID),
CONSTRAINT UQ_Takmicenja UNIQUE(Naziv,SezonaID),
CONSTRAINT CHK_Takmicenja_Datumi CHECK(DatumZavrsetka>=DatumPocetka),
CONSTRAINT CHK_Takmicenja_Status CHECK(StatusTakmicenja IN(N'Planirano',N'U
toku',N'Završeno',N'Otkazano'))),
```

```
CONSTRAINT CHK_Takmicenja_Rang CHECK(RangTakmicenja  
IN(N'Lokalno',N'Državno',N'Regionalno',N'Međunarodno')),  
CONSTRAINT CHK_Takmicenja_Fond CHECK(NagradniFond>=0)
```

2.1.7. Tabela Ekipe

Tabela Ekipe čuva podatke o sportskim ekipama. EkipaID je primarni ključ sa identity svojstvom. SportID i GradID su strani ključevi. Naziv i SkraceniNaziv su obavezni i jedinstveni. DatumOsnivanja je opcionalan, dok Budzet može biti NULL, a ako nije NULL, mora biti veći ili jednak nuli. Aktivan je logičko polje sa podrazumijevanom vrijednošću 1.

```
EkipaID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Ekipe PRIMARY KEY,  
SportID INT NOT NULL,  
GradID INT NOT NULL,  
Naziv NVARCHAR(100) NOT NULL CONSTRAINT UQ_Ekipe_Naziv UNIQUE,  
SkraceniNaziv NVARCHAR(10) NOT NULL CONSTRAINT UQ_Ekipe_Skraceni UNIQUE,  
DatumOsnivanja DATE NULL,  
Budzet DECIMAL(14,2) NULL,  
Aktivan BIT NOT NULL CONSTRAINT DF_Ekipe_Aktivan DEFAULT 1,  
CONSTRAINT FK_Ekipe_Sportovi FOREIGN KEY(SportID) REFERENCES  
sif.Sportovi(SportID),  
CONSTRAINT FK_Ekipe_Gradovi FOREIGN KEY(GradID) REFERENCES  
sif.Gradovi(GradID),  
CONSTRAINT CHK_Ekipe_Budzet CHECK(Budzet IS NULL OR Budzet>=0)
```

2.1.8. Tabela Sportisti

Tabela Sportisti čuva podatke o sportistima. SportistaID je primarni ključ sa identity svojstvom. DrzavaID je strani ključ. Ime, Prezime i DatumRodjenja su obavezna polja. Pol je opcion, sa dozvoljenim vrijednostima M ili Z. Email je opcion i jedinstven. VisinaCm i TezinaKg su opcioni sa CHECK ograničenjima koja definišu realne opsege. DatumRegistracije ima podrazumijevanu vrijednost GETDATE().

```

SportistaID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Sportisti PRIMARY KEY,
DrzavaID INT NOT NULL,
Ime NVARCHAR(50) NOT NULL,
Prezime NVARCHAR(50) NOT NULL,
DatumRodjenja DATE NOT NULL,
Pol NCHAR(1) NULL,
Email NVARCHAR(100) NULL,
VisinaCm DECIMAL(5,2) NULL,
TezinaKg DECIMAL(5,2) NULL,
DatumRegistracije DATETIME NOT NULL CONSTRAINT DF_Sportisti_Registracija
DEFAULT GETDATE(),
Aktivan BIT NOT NULL CONSTRAINT DF_Sportisti_Aktivan DEFAULT 1,
CONSTRAINT FK_Sportisti_Drzave FOREIGN KEY(DrzavaID) REFERENCES
sif.Drzave(DrzavaID),
CONSTRAINT UQ_Sportisti_Email UNIQUE>Email),
CONSTRAINT CHK_Sportisti_Pol CHECK(Pol IS NULL OR Pol IN(N'M',N'Z')),
CONSTRAINT CHK_Sportisti_Email CHECK>Email IS NULL OR Email LIKE N'%@%._%'),
CONSTRAINT CHK_Sportisti_Visina CHECK(VisinaCm IS NULL OR VisinaCm BETWEEN
100 AND 250),
CONSTRAINT CHK_Sportisti_Tezina CHECK(TezinaKg IS NULL OR TezinaKg BETWEEN
30 AND 250)

```

2.1.9. Tabela Utakmice

Tabela Utakmice je srce takmičarskog dijela sistema. UtakmicaID je primarni ključ sa identity svojstvom. TakmicenjeID, ObjekatID i FazaID su strani ključevi. DomacaEkipaID, GostjucaEkipaID i SudijaID su opcioni strani ključevi. DatumVrijeme je obavezan. StatusUtakmice može imati vrijednosti Zakazana, U toku, Završena, Odložena ili Otkazana. RezultatDomaci i RezultatGosti su opcioni, ali ako su navedeni, oba moraju biti nenegativna. BrojGledalaca je opcion sa ograničenjem da mora biti veći ili jednak nuli.

```

UtakmicaID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Utakmice PRIMARY KEY,
TakmicenjeID INT NOT NULL,
FazaID INT NULL,
ObjekatID INT NOT NULL,
DomacaEkipaID INT NULL,
GostujucaEkipaID INT NULL,
SudijaID INT NULL,
DatumVrijeme DATETIME NOT NULL,
StatusUtakmice NVARCHAR(20) NOT NULL CONSTRAINT DF_Utakmice_Status
DEFAULT N'Zakazana',
RezultatDomaci INT NULL,
RezultatGosti INT NULL,
BrojGledalaca INT NULL,
Napomena NVARCHAR(300) NULL,
CONSTRAINT FK_Utakmice_Takmicenja FOREIGN KEY(TakmicenjeID) REFERENCES
sport.Takmicenja(TakmicenjeID),
CONSTRAINT FK_Utakmice_Faze FOREIGN KEY(FazaID) REFERENCES
sport.Faze(FazaID),
CONSTRAINT FK_Utakmice_Objekti FOREIGN KEY(ObjekatID) REFERENCES
sport.Objekti(ObjekatID),
CONSTRAINT FK_Utakmice_Domaci FOREIGN KEY(DomacaEkipaID) REFERENCES
sport.Ekipe(EkipaID),
CONSTRAINT FK_Utakmice_Gosti FOREIGN KEY(GostujucaEkipaID) REFERENCES
sport.Ekipe(EkipaID),
CONSTRAINT FK_Utakmice_Sudije FOREIGN KEY(SudijaID) REFERENCES
sport.Sudije(SudijaID),
CONSTRAINT CHK_Utakmice_Ekipe CHECK(DomacaEkipaID IS NULL OR
GostujucaEkipaID IS NULL OR DomacaEkipaID<>GostujucaEkipaID),
CONSTRAINT CHK_Utakmice_Status CHECK(StatusUtakmice IN(N'Zakazana',N'U
toku',N'Završena',N'Odložena',N'Otkazana')),

```

```
CONSTRAINT CHK_Utakmice_Rezultat CHECK((RezultatDomaci IS NULL AND
RezultatGosti IS NULL) OR (RezultatDomaci>=0 AND RezultatGosti>=0)),
CONSTRAINT CHK_Utakmice_Gledaoci CHECK(BrojGledalaca IS NULL OR
BrojGledalaca>=0)
```

2.1.10. Tabela Ulaznice

Tabela Ulaznice služi za evidenciju karata za utakmice. UlaznicaID je primarni ključ sa identity svojstvom. UtakmicaID je strani ključ. Sektor, RedBroj i SjedisteBroj definišu lokaciju sjedišta, sa UNIQUE ograničenjem za kombinaciju utakmice, sektora, reda i broja sjedišta. Cijena je obavezna i mora biti veća ili jednaka nuli. DatumProdaje je opcion. StatusUlaznice može biti Slobodna, Rezervisana, Prodata ili Ponistena. KupacEmail je opcion.

```
UlaznicaID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Ulaznice PRIMARY KEY,
UtakmicaID INT NOT NULL,
Sektor NVARCHAR(20) NOT NULL,
RedBroj NVARCHAR(10) NOT NULL,
SjedisteBroj INT NOT NULL,
Cijena DECIMAL(10,2) NOT NULL,
DatumProdaje DATETIME NULL,
StatusUlaznice NVARCHAR(20) NOT NULL CONSTRAINT DF_Ulaznice_Status DEFAULT
N'Slobodna',
KupacEmail NVARCHAR(100) NULL,
CONSTRAINT FK_Ulaznice_Utakmice FOREIGN KEY(UtakmicaID) REFERENCES
sport.Utakmice(UtakmicaID),
CONSTRAINT UQ_Ulaznice_Sjediste UNIQUE(UtakmicaID,Sektor,RedBroj,SjedisteBroj),
CONSTRAINT CHK_Ulaznice_Cijena CHECK(Cijena>=0),
CONSTRAINT CHK_Ulaznice_Status CHECK(StatusUlaznice
IN(N'Slobodna',N'Rezervisana',N'Prodata',N'Ponistena'))
```


2.1.11. Tabela Placanja

Tabela Placanja služi za evidentiranje uplata za ulaznice. PlacanjeID je primarni ključ sa identity svojstvom. UlaznicaID je strani ključ. DatumPlacanja ima podrazumijevanu vrijednost GETDATE(). Iznos je obavezan i mora biti veći od nule. NacinPlacanja može imati vrijednosti Gotovina, Kartica, Online ili Vaucer. StatusPlacanja može imati vrijednosti Na cekanju, Placeno, Odbijeno ili Refundirano. BrojTransakcije je opcion sa jedinstvenim filtriranim indeksom.

```
PlacanjeID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_Placanja PRIMARY KEY,  
UlaznicaID INT NOT NULL,  
DatumPlacanja DATETIME NOT NULL CONSTRAINT DF_Placanja_Datum DEFAULT  
GETDATE(),  
Iznos DECIMAL(10,2) NOT NULL,  
NacinPlacanja NVARCHAR(20) NOT NULL,  
StatusPlacanja NVARCHAR(20) NOT NULL CONSTRAINT DF_Placanja_Status DEFAULT  
N'Placeno',  
BrojTransakcije NVARCHAR(60) NULL,  
CONSTRAINT FK_Placanja_Ulaznice FOREIGN KEY(UlaznicaID) REFERENCES  
prodaja.Ulaznice(UlaznicaID),  
CONSTRAINT CHK_Placanja_Iznos CHECK(Iznos>0),  
CONSTRAINT CHK_Placanja_Nacin CHECK(NacinPlacanja  
IN(N'Gotovina',N'Kartica',N'Online',N'Vaucer'))),  
CONSTRAINT CHK_Placanja_Status CHECK(StatusPlacanja IN(N'Na  
cekanju',N'Placeno',N'Odbijeno',N'Refundirano'))
```

2.1.12. Tabela za praćenje promjena utakmica

Tabela UtakmiceDnevnik služi za evidentiranje svih promjena nad utakmicama. DnevnikID je primarni ključ sa identity svojstvom. UtakmicaID bilježi na koju utakmicu se promjena odnosi. Akcija može biti INSERT, UPDATE ili DELETE. StariStatus i NoviStatus prate promjene statusa, dok StariRezultat i NoviRezultat prate promjene rezultata. DatumAkcije ima podrazumijevanu vrijednost GETDATE(), a Korisnik bilježi ko je izvršio promjenu korištenjem SUSER_SNAME().

```
DnevnikID INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_UtakmiceDnevnik
PRIMARY KEY,
UtakmicaID INT NOT NULL,
Akcija NVARCHAR(10) NOT NULL,
StariStatus NVARCHAR(20) NULL,
NoviStatus NVARCHAR(20) NULL,
StariRezultat NVARCHAR(20) NULL,
NoviRezultat NVARCHAR(20) NULL,
DatumAkcije DATETIME NOT NULL CONSTRAINT DF_UD_Datum DEFAULT
GETDATE(),
Korisnik NVARCHAR(128) NOT NULL CONSTRAINT DF_UD_Korisnik DEFAULT
SUSER_SNAME()
```

2.2. USPOSTAVLJANJE INTEGRITETA NAD PODACIMA

Integritet podataka predstavlja jedan od najvažnijih koncepata u projektovanju baza podataka. On osigurava da su podaci u bazi tačni, konzistentni i da zadovoljavaju sve poslovne zahtjeve. U ovoj bazi podataka uspostavljena su tri nivoa integriteta: integritet entiteta, integritet domena i referencijalni integritet.

2.2.1. Integritet na nivou entiteta

Integritet entiteta osigurava da svaki zapis u tabeli bude jedinstven i jasno identifikovan, čime se sprečava pojava duplih podataka koji bi mogli predstavljati isti entitet. U bazi podataka SportskaTakmicenja, integritet entiteta postiže se definisanjem primarnih ključeva za sve glavne tabele. Primarni ključevi su postavljeni na kolone DrzavaID, GradID, SportID, OrganizatorID, SezonaID, TakmicenjeID, FazaID, EkipaID, SportistaID, ClanEkipeID, ObjekatID, SudijaID, UtakmicaID, NastupID, KaznaID, SponzorID, UlaznicaID, PlacanjeID, DnevnikID iz audit tabela i druge. Većina ovih kolona koristi IDENTITY svojstvo, što omogućava automatsko generisanje jedinstvenih vrijednosti pri unosu novih zapisa.

2.2.2. Integritet na nivou domena

Integritet domena se odnosi na ograničenja koja definišu dozvoljene vrijednosti za pojedine kolone. Ovaj oblik integriteta osigurava da podaci unutar baze budu validni i konzistentni. U praksi se postiže korišćenjem CHECK ograničenja, koja postavljaju konkretna pravila za unos podataka. U tabeli Sportovi, CHECK ograničenja definišu dozvoljene tipove sportova i osiguravaju da je maksimalan broj igrača veći ili jednak minimalnom. U tabeli Sportisti, CHECK ograničenja definišu dozvoljene vrijednosti za pol, opseg visine i težine, te format email adrese. U tabeli Takmicenja, CHECK ograničenja definišu dozvoljene statuse i rangove takmičenja, te osiguravaju da je nagradni fond nenegativan.

2.2.3. Ograničenja za tabelu Utakmice

Za tabelu Utakmice, uspostavljena su četiri CHECK ograničenja. Prvo ograničenje osigurava da domaća i gostujuća ekipa ne budu iste. Drugo ograničenje definiše dozvoljene statuse utakmice. Treće ograničenje provjerava da su rezultati, ako su uneseni, nenegativni. Četvrto ograničenje osigurava da broj gledalaca, ako je unesen, bude nenegativan.

```
CONSTRAINT CHK_Utakmice_Ekipe CHECK(DomacaEkipaID IS NULL OR  
GostujucaEkipaID IS NULL OR DomacaEkipaID <> GostujucaEkipaID),  
CONSTRAINT CHK_Utakmice_Status CHECK(StatusUtakmice IN(N'Zakazana',N'U  
toku',N'Završena',N'Odložena',N'Otkazana')),  
CONSTRAINT CHK_Utakmice_Rezultat CHECK((RezultatDomaci IS NULL AND  
RezultatGosti IS NULL) OR (RezultatDomaci >=0 AND RezultatGosti >=0)),  
CONSTRAINT CHK_Utakmice_Gledaoci CHECK(BrojGledalaca IS NULL OR  
BrojGledalaca >=0)
```

2.2.4. Ograničenja za tabelu Placanja

Za tabelu Placanja, uspostavljena su tri CHECK ograničenja. Prvo ograničenje provjerava da je iznos uplate veći od nule. Drugo ograničenje definiše dozvoljene načine plaćanja. Treće ograničenje definiše dozvoljene statuse plaćanja.

```
CONSTRAINT CHK_Placanja_Iznos CHECK(Iznos>0),  
CONSTRAINT CHK_Placanja_Nacin CHECK(NacinPlacanja  
IN(N'Gotovina',N'Kartica',N'Online',N'Vaucer')),  
CONSTRAINT CHK_Placanja_Status CHECK(StatusPlacanja IN(N'Na  
cekanju',N'Placeno',N'Odbijeno',N'Refundirano'))
```

2.2.5. Referencijalni integritet u bazi

Referencijalni integritet osigurava da veze između tabela budu konzistentne. U bazi SportskaTakmicenja, referencijalni integritet je uspostavljen kroz više stranih ključeva. Tabela Gradovi sadrži strani ključ DrzavaID koji referencira tabelu Drzave. Tabela Takmicenja sadrži tri strana ključa, SportID koji referencira tabelu Sportovi, OrganizatorID koji referencira tabelu Organizatori i SezonaID koji referencira tabelu Sezone.

Tabela Ekipe sadrži dva strana ključa. SportID referencira tabelu Sportovi, a GradID referencira tabelu Gradovi. Tabela Sportisti sadrži strani ključ DrzavaID koji referencira tabelu Drzave. Tabela Utakmice sadrži više stranih ključeva, uključujući TakmicenjeID, FazaID, ObjekatID, DomacaEkipaID, GostujucaEkipaID i SudijaID. Tabela ClanoviEkipa sadrži dva strana ključa, EkipaID koji referencira tabelu Ekipe i SportistaID koji referencira tabelu Sportisti. Tabela Nastupi sadrži tri strana ključa, UtakmicaID, SportistaID i EkipaID. Tabela Ulaznice sadrži strani ključ UtakmicaID, dok Placanja sadrže strani ključ UlaznicaID. Primjer nevažećeg unosa koji bi narušio referencijalni integritet je pokušaj unosa ekipe sa nepostojećim sportom. Ovakav unos bi rezultirao greškom jer SQL Server ne dozvoljava unos vrijednosti koje ne postoje u nadređenoj tabeli.

2.3. Kreiranje indeksa

Indeksi su posebne strukture u bazi podataka koje značajno ubrzavaju izvršavanje upita, posebno onih koji pretražuju, filtriraju ili sortiraju velike količine podataka. Razlog zašto indeksi ubrzavaju pretragu sličan je razlogu zašto indeks na kraju knjige pomaže da brzo pronađemo željeno poglavlje umjesto da listamo cijelu knjigu.

2.3.1. Indeks na kolonama DrzavaID i Naziv u tabeli Gradovi

Prvi indeks je kreiran na kolonama DrzavaID i Naziv u tabeli Gradovi. Ovaj indeks je važan jer će se gradovi često pretraživati po državi i nazivu prilikom povezivanja sa ekipama i objektima. Kompozitni indeks omogućava brzo pronalaženje gradova za određenu državu.

```
CREATE INDEX IX_Gradovi_Drzava ON sif.Gradovi(DrzavaID,Naziv)
```

2.3.2. Indeks na kolonama SportID i SezonaID u tabeli Takmicenja

Drugi indeks je kreiran na kolonama SportID i SezonaID u tabeli Takmicenja. Ovaj indeks je važan za upite koji pretražuju takmičenja po sportu i sezoni, što je česta operacija prilikom pregleda rasporeda takmičenja.

```
CREATE INDEX IX_Takmicenja_SportSezona ON sport.Takmicenja(SportID,SezonaID)
```

2.3.3. Indeks na kolonama Prezime i Ime u tabeli Sportisti

Treći indeks je kreiran na kolonama Prezime i Ime u tabeli Sportisti. Ovaj indeks je izuzetno važan jer će se sportisti često pretraživati po prezimenu i imenu prilikom pretrage ili povezivanja sa nastupima.

```
CREATE INDEX IX_Sportisti_PrezimeIme ON sport.Sportisti(Prezime,Ime)
```

2.3.4. Indeks na kolonama EkipaID i DatumDo u tabeli ClanoviEkipa

Četvrti indeks je kreiran na kolonama EkipaID i DatumDo u tabeli ClanoviEkipa. Ovaj indeks je važan za upite koji pretražuju aktivne članove ekipa, gdje se DatumDo koristi za filtriranje trenutnih članstava.

```
CREATE INDEX IX_Clanovi_Ekipa ON sport.ClanoviEkipa(EkipaID,DatumDo)
```

2.3.5. Indeks na koloni DatumVrijeme u tabeli Utakmice

Peti indeks je kreiran na koloni DatumVrijeme u tabeli Utakmice. Ovaj indeks je važan jer će sistem često izvršavati upite koji pronalaze utakmice za određeni period, kao što je prikaz rasporeda za određeni dan ili sedmicu.

```
CREATE INDEX IX_Utakmice_Datum ON sport.Utakmice(DatumVrijeme)
```

2.3.6. Kompozitni indeks na kolonama TakmicenjeID i StatusUtakmice u tabeli Utakmice

Šesti indeks je kompozitni indeks na kolonama TakmicenjeID i StatusUtakmice u tabeli Utakmice. Ovaj indeks je namijenjen upitima koji pretražuju utakmice za određeno takmičenje sa određenim statusom.

```
CREATE INDEX IX_Utakmice_TakmicenjeStatus ON  
sport.Utakmice(TakmicenjeID,StatusUtakmice)
```

2.3.7. Indeks na koloni SportistaID u tabeli Nastupi

Sedmi indeks je kreiran na koloni SportistaID u tabeli Nastupi. Ovaj indeks je važan za upite koji pretražuju nastupe određenog sportiste, što je često za statistiku.

```
CREATE INDEX IX_Nastupi_Sportista ON sport.Nastupi(SportistaID)
```

2.3.8. Kompozitni indeks na kolonama StatusUlaznice i UtakmicaID u tabeli Ulaznice

Osmi indeks je kompozitni indeks na kolonama StatusUlaznice i UtakmicaID u tabeli Ulaznice. Ovaj indeks je važan za upite koji pretražuju slobodne ili prodate ulaznice za određenu utakmicu.

```
CREATE INDEX IX_Ulaznice_Status ON prodaja.Ulaznice(StatusUlaznice,UtakmicaID)
```

2.3.9. Indeks na koloni DatumPlacanja u tabeli Placanja

Deveti indeks je kreiran na koloni DatumPlacanja u tabeli Placanja. Ovaj indeks je važan za finansijske izvještaje koji analiziraju prihode po vremenskim periodima.

```
CREATE INDEX IX_Placanja_Datum ON prodaja.Placanja(DatumPlacanja)
```

2.3.10. Filtrirani jedinstveni indeks na koloni BrojTransakcije u tabeli Placanja

Deseti indeks je filtrirani jedinstveni indeks na koloni BrojTransakcije u tabeli Placanja. Ovaj indeks je poseban jer se primjenjuje samo na redove gdje BrojTransakcije nije NULL. Ovakav indeks sprečava dupliranje brojeva transakcija, što je važno za praćenje online plaćanja, dok istovremeno dozvoljava više NULL vrijednosti.

```
CREATE UNIQUE INDEX UX_Placanja_BrojTransakcije ON  
prodaja.Placanja(BrojTransakcije) WHERE BrojTransakcije IS NOT NULL
```

2.4. Unos podataka u tabele

Nakon što je struktura baze podataka u potpunosti definisana, potrebno je unijeti test podatke koji će omogućiti demonstraciju funkcionalnosti sistema. Test podaci su odabrani tako da pokriju različite scenarije koji se mogu pojaviti u stvarnom radu sistema.

2.4.1. Unos podataka u tabelu Drzave

U tabelu Drzave unosi se osam država koje pokrivaju područje jugoistočne Evrope. Unose se Bosna i Hercegovina, Srbija, Hrvatska, Slovenija, Crna Gora, Austrija, Italija i Njemačka. Svaka država ima svoj trocifreni ISO kod.

```
INSERT INTO sif.Drzave(Naziv,ISOKod) VALUES  
(N'Bosna i Hercegovina',N'BIH'),(N'Srbija',N'SRB'),(N'Hrvatska',N'HRV'),(N'Slovenija',N'SVN'),  
(N'Crna Gora',N'MNE'),(N'Austrija',N'AUT'),(N'Italija',N'ITA'),(N'Njemačka',N'DEU');
```

Slika 3 - Unos podataka u tabelu Drzave

2.4.2. Unos podataka u tabelu Gradovi

U tabelu Gradovi unosi se dvanaest gradova raspoređenih po različitim državama. Iz Bosne i Hercegovine unose se Banja Luka, Sarajevo, Mostar i Tuzla. Iz Srbije se unose Beograd i Novi Sad. Iz Hrvatske se unose Zagreb i Split. Iz Slovenije se unosi Ljubljana, iz Crne Gore Podgorica, iz Austrije Beč i iz Italije Milano. Svaki grad ima svoj poštanski broj i broj stanovnika.

```
INSERT INTO sif.Gradovi(DrzavaID,Naziv,PostanskiBroj,BrojStanovnika) VALUES
(1,N'Banja Luka',N'78000',185000),(1,N'Sarajevo',N'71000',275000),(1,N'Mostar',N'88000',106000),
(1,N'Tuzla',N'75000',111000),(2,N'Beograd',N'11000',1370000),(2,N'Novi Sad',N'21000',370000),
(3,N'Zagreb',N'10000',770000),(3,N'Split',N'21000',160000),(4,N'Ljubljana',N'1000',295000),
(5,N'Podgorica',N'81000',190000),(6,N'Beč',N'1010',200000),(7,N'Milano',N'20100',1360000);
```

Slika 4 - Unos podataka u tabelu Gradovi

2.4.3. Unos podataka u tabelu Sportovi

U tabelu Sportovi unosi se osam različitih sportova. Unose se Fudbal, Košarka, Rukomet i Odbojka kao ekipni sportovi, te Tenis, Atletika, Plivanje i Stoni tenis kao individualni sportovi. Za svaki sport definisan je minimalan i maksimalan broj igrača.

```
INSERT INTO sif.Sportovi(Naziv,TipSporta,MinimalanBrojIgraca,MaksimalanBrojIgraca) VALUES
(N'Fudbal',N'Ekipni',11,23),(N'Košarka',N'Ekipni',5,15),(N'Rukomet',N'Ekipni',7,16),(N'Odbojka',N'Ekipni',6,14),
(N'Tenis',N'Individualni',1,2),(N'Atletika',N'Individualni',1,1),(N'Plivanje',N'Individualni',1,1),(N'Stoni tenis',N'Individualni',1,2);
```

Slika 5 - Unos podataka u tabelu Sportovi

2.4.4. Unos podataka u tabelu Organizatori

U tabelu Organizatori unosi se pet organizatora. Unose se Sportski savez BiH, Fudbalski savez RS, Košarkaški savez BiH, Atletski savez BiH i Tenis asocijacija regiona. Svaki organizator ima svoj JIB, email, telefon i datum osnivanja.

```
INSERT INTO sport.Organizatori(Naziv,JIB,Email,Telefon,DatumOsnivanja) VALUES
(N'Sportski savez BiH',N'420000000001',N'kontakt@ssbih.example',N'033100100',N'1992-06-01'),
(N'Fudbalski savez RS',N'440000000002',N'info@fsrs.example',N'051200200',N'1992-09-05'),
(N'Košarkaški savez BiH',N'420000000003',N'office@ksbih.example',N'033300300',N'1950-01-01'),
(N'Atletski savez BiH',N'420000000004',N'info@asbih.example',N'033400400',N'1946-01-01'),
(N'Tenis asocijacija regiona',N'440000000005',N'kontakt@tar.example',N'051500500',N'2001-04-12');
```

Slika 6 - Unos podataka u tabelu Organizatori

2.4.5. Unos podataka u tabelu Sezone

U tabelu Sezone unose se četiri sezone. Unose se sezone 2024/2025, 2025/2026 i 2026/2027 sa datumima početka u augustu i završetka u julu, te kalendarska 2026 koja traje od januara do decembra. Aktivna sezona je 2026/2027.


```
INSERT INTO sport.Sezone(Naziv,DatumPocetka,DatumZavrsetka,Aktivna) VALUES
(N'2024/2025','2024-08-01','2025-07-31',0),(N'2025/2026','2025-08-01','2026-07-31',0),
(N'2026/2027','2026-08-01','2027-07-31',1),(N'Kalendarska 2026','2026-01-01','2026-12-31',1);
```

Slika 7 - Unos podataka u tabelu Sezone

2.4.6. Unos podataka u tabelu Takmicenja

U tabelu Takmicenja unosi se šest takmičenja. Unose se Premijer liga Republike Srpske za fudbal, Košarkaška liga BiH, Regionalni rukometni kup, Odbojkaški kup gradova, Banja Luka Open 2026 za tenis i Atletski miting BiH 2026. Svako takmičenje ima svoj rang, datume početka i završetka, status i nagradni fond.

```
INSERT INTO sport.Takmicenja(SportID,OrganizatorID,SezonaID,Naziv,RangTakmicenja,DatumPocetka,DatumZavrsetka,StatusTakmicenja,NagradniFond) VALUES
(1,2,3,N'Premijer liga Republike Srpske',N'Državno','2026-08-15','2027-05-30',N'U toku',150000.00),
(2,3,3,N'Košarkaška liga BiH',N'Državno','2026-09-01','2027-05-15',N'Planirano',120000.00),
(3,1,3,N'Regionalni rukometni kup',N'Regionalno','2026-10-01','2026-10-15',N'Planirano',50000.00),
(4,1,3,N'Odbojkaški kup gradova',N'Regionalno','2026-11-01','2026-11-20',N'Planirano',35000.00),
(5,5,4,N'Banja Luka Open 2026',N'Medunarodno','2026-06-10','2026-06-17',N'Završeno',80000.00),
(6,4,4,N'Atletski miting BiH 2026',N'Medunarodno','2026-09-20','2026-09-21',N'Planirano',25000.00);
```

Slika 8 - Unos podataka u tabelu Takmicenja

2.4.7. Unos podataka u tabelu Faze

U tabelu Faze unose se faze za svako takmičenje. Za Premijer ligu unose se Ligaški dio i Završnica. Za Košarkašku ligu unose se Regularna sezona i Play-off. Za Rukometni kup unose se Grupna faza i Eliminacije. Za Odbojkaški kup unose se Grupe i Finalni turnir. Za Banja Luka Open unose se Kvalifikacije i Glavni žrijeb. Za Atletski miting unose se Kvalifikacije i Finala.

```
INSERT INTO sport.Faze(TakmicenjeID,Naziv,RedniBroj,DatumPocetka,DatumZavrsetka) VALUES
(1,N'Ligaški dio',1,'2026-08-15','2027-04-30'),(1,N'Završnica',2,'2027-05-01','2027-05-30'),
(2,N'Regularna sezona',1,'2026-09-01','2027-04-15'),(2,N'Play-off',2,'2027-04-20','2027-05-15'),
(3,N'Grupna faza',1,'2026-10-01','2026-10-08'),(3,N'Eliminacije',2,'2026-10-09','2026-10-15'),
(4,N'Grupe',1,'2026-11-01','2026-11-10'),(4,N'Finalni turnir',2,'2026-11-15','2026-11-20'),
(5,N'Kvalifikacije',1,'2026-06-10','2026-06-11'),(5,N'Glavni žrijeb',2,'2026-06-12','2026-06-17'),
(6,N'Kvalifikacije',1,'2026-09-20','2026-09-20'),(6,N'Finala',2,'2026-09-21','2026-09-21');
```

Slika 9 - Unos podataka u tabelu Faze

2.4.8. Unos podataka u tabelu Ekipe

U tabelu Ekipe unosi se dvanaest ekipa raspoređenih po različitim sportovima i gradovima. Za fudbal unose se FK Borac Banja Luka, FK Sarajevo, FK Crvena zvezda i GNK Zagreb. Za košarku unose se KK Borac, KK Bosna, KK Beograd i KK Zagreb. Za rukomet unose se RK Borac i RK Mostar. Za odbojku unose se OK Tuzla i OK Novi Sad. Svaka ekipa ima svoj skraćeni naziv, datum osnivanja i budžet.

```
INSERT INTO sport.Ekipe(SportID, GradID, Naziv, SkraceniNaziv, DatumOsnivanja, Budzet) VALUES
(1,1,N'FK Borac Banja Luka',N'BOR','1926-07-04',3500000),(1,2,N'FK Sarajevo',N'SAR','1946-10-24',4200000),
(1,5,N'FK Crvena zvezda',N'CZV','1945-03-04',18000000),(1,7,N'GNK Zagreb',N'ZG','1911-04-26',9000000),
(2,1,N'KK Borac',N'KKB','1947-01-01',850000),(2,2,N'KK Bosna',N'BOS','1951-01-01',1100000),
(2,5,N'KK Beograd',N'KBG','1945-01-01',1600000),(2,7,N'KK Zagreb',N'KZG','1946-01-01',1400000),
(3,1,N'RK Borac',N'RKB','1950-01-01',700000),(3,3,N'RK Mostar',N'RKM','1952-01-01',500000),
(4,4,N'OK Tuzla',N'OKT','1960-01-01',350000),(4,6,N'OK Novi Sad',N'ONS','1958-01-01',600000);
```

Slika 10 - Unos podataka u tabelu Ekipe

2.4.9. Unos podataka u tabelu Sportisti

U tabelu Sportisti unosi se trideset dva sportista sa različitim karakteristikama. Sportisti su raspoređeni po različitim državama, sa različitim datumima rođenja, polom, visinom i težinom. Svaki sportista ima svoj email koji je jedinstven. Sportisti pokrivaju različite sportske discipline i omogućavaju testiranje različitih upita.

```
INSERT INTO sport.Sportisti(DrzavaID, Ime, Prezime, DatumRodjenja, Pol, Email, VisinaCm, TezinaKg) VALUES
(2,N'Marko',N'Marković','1988-02-02',N'M',N'sportista01@example.com',171.00,61.00),
(3,N'Nikola',N'Petrović','1989-03-03',N'M',N'sportista02@example.com',172.00,62.00),
(4,N'Stefan',N'Jovanović','1990-04-04',N'M',N'sportista03@example.com',173.00,63.00),
(5,N'Luka',N'Ilić','1991-05-05',N'M',N'sportista04@example.com',174.00,64.00),
(6,N'Aleksandar',N'Savić','1992-06-06',N'M',N'sportista05@example.com',175.00,65.00),
(7,N'Milan',N'Kovačević','1993-07-07',N'M',N'sportista06@example.com',176.00,66.00),
(8,N'Nemanja',N'Pavlović','1994-08-08',N'M',N'sportista07@example.com',177.00,67.00),
(1,N'Ognjen',N'Marić','1995-09-09',N'M',N'sportista08@example.com',178.00,68.00),
(2,N'Ivan',N'Nikolić','1996-10-10',N'M',N'sportista09@example.com',179.00,69.00),
(3,N'Petar',N'Popović','1997-11-11',N'M',N'sportista10@example.com',180.00,70.00),
(4,N'Bojan',N'Radovanović','1998-12-12',N'M',N'sportista11@example.com',181.00,71.00),
(5,N'Dejan',N'Tomić','1999-01-13',N'M',N'sportista12@example.com',182.00,72.00),
```

Slika 11 - Unos podataka u tabelu Sportisti

2.4.10. Unos podataka u tabelu ClanoviEkipa

U tabelu ClanoviEkipa unose se članstva sportista u ekipama. Svaki sportista je dodijeljen jednoj ekipi, sa brojem dresa i pozicijom. DatumOd je postavljen na 1. august 2026. godine, dok je DatumDo NULL, što znači da su članstva trenutno aktivna.

```
INSERT INTO sport.ClanoviEkipa(EkipaID, SportistaID, BrojDresa, Pozicija, DatumOd, DatumDo) VALUES
(1,1,4,N'Igrač','2026-08-01',NULL),
(1,2,7,N'Igrač','2026-08-01',NULL),
(2,3,10,N'Igrač','2026-08-01',NULL),
(2,4,13,N'Igrač','2026-08-01',NULL),
```

Slika 12 - Unos podataka u tabelu ClanoviEkipa

2.4.11. Unos podataka u tabelu Objekti

U tabelu Objekti unosi se deset sportskih objekata raspoređenih po različitim gradovima. Unose se stadioni, sportske dvorane, arene i teniski centri. Svaki objekat ima svoj kapacitet, tip i datum

otvaranja. Objekti uključuju Gradski stadion Banja Luka, SD Borik, Stadion Koševo, Skenderiju, Dvoranu Mostar, Mejdan, Beogradsku arenu, SPENS, Arenu Zagreb i Teniski centar Mladost.

```
INSERT INTO sport.Objekti(GradID,Naziv,Adresa,Kapacitet,TipObjekta,DatumOtvaranja) VALUES
(1,N'Gradski stadion Banja Luka',N'Vladike Platona 6',10030,N'Stadion','1937-09-05'),
(1,N'SD Borik',N'Aleja Svetog Save 48',3000,N'Sportska dvorana','1974-04-20'),
(2,N'Stadion Koševo',N'Patriotske lige 35',34500,N'Stadion','1947-01-01'),
(2,N'Skenderija',N'Terezija bb',6500,N'Sportska dvorana','1969-11-29'),
(3,N'Dvorana Mostar',N'Kneza Višeslava bb',3500,N'Sportska dvorana','1995-01-01'),
(4,N'Mejdan',N'Bosne Srebrene bb',5000,N'Sportska dvorana','1984-10-02'),
(5,N'Beogradska arena',N'Bulevar Arsenija Čarnojevića 58',18386,N'Arena','2004-07-31'),
(6,N'SPENS',N'Sutjeska 2',11000,N'Sportski centar','1981-04-14'),
(7,N'Arena Zagreb',N'Ulica Vice Vukova 8',15200,N'Arena','2008-12-27'),
(1,N'Teniski centar Mladost',N'Park Mladen Stojanović bb',2500,N'Teniski centar','2002-05-01');
```

Slika 13 - Unos podataka u tabelu Objekti

2.4.12. Unos podataka u tabelu Sudije

U tabelu Sudije unosi se osam sudija sa različitim državama porekla. Svaka sudija ima svoj broj licence, datum licence i kategoriju koja može biti Međunarodna, Nacionalna ili Regionalna.

```
INSERT INTO sport.Sudije(DrzavaID,Ime,Prezime,BrojLicence,DatumLicence,Kategorija) VALUES
(1,N'Damir',N'Kovač',N'LIC-001','2018-01-15',N'Međunarodna'),(2,N'Miloš',N'Janković',N'LIC-002','2017-03-20',N'Nacionalna'),
(3,N'Ivan',N'Horvat',N'LIC-003','2019-06-10',N'Međunarodna'),(1,N'Adnan',N'Hodžić',N'LIC-004','2020-02-12',N'Nacionalna'),
(4,N'Matej',N'Kranjc',N'LIC-005','2016-09-01',N'Međunarodna'),(5,N'Marko',N'Vukčević',N'LIC-006','2021-04-22',N'Regionalna'),
(6,N'Johann',N'Mayer',N'LIC-007','2015-05-11',N'Međunarodna'),(7,N'Luca',N'Rossi',N'LIC-008','2014-08-19',N'Međunarodna');
```

Slika 14 - Unos podataka u tabelu Sudije

2.4.13. Unos podataka u tabelu Utakmice

U tabelu Utakmice unosi se dvadeset četiri utakmice raspoređenih po različitim takmičenjima i terminima. Prvih osam utakmica je završeno sa evidentiranim rezultatima, dok su preostale zakazane. Utakmice pokrivaju različite kombinacije ekipa, objekata i sudija.

```
INSERT INTO sport.Utakmice(TakmicenjeID,FazaID,ObjekatID,DomacaEkipaID,GostujucaEkipaID,SudijaID,DatumVrijeme,StatusUtakmice,RezultatDomaci,RezultatGosti,BrojGledalaca,Napomena) VALUES
(1,1,1,1,2,1,'2026-09-01T18:00:00',N'Završena',0,1,1200,NULL),
(1,1,3,2,3,2,'2026-09-02T19:00:00',N'Završena',2,0,1373,NULL),
(1,1,7,3,4,3,'2026-09-03T20:00:00',N'Završena',4,3,1546,NULL),
(1,1,9,4,1,4,'2026-09-04T18:00:00',N'Završena',1,2,1719,NULL),
(1,1,2,1,2,5,'2026-09-05T19:00:00',N'Završena',3,1,1892,NULL),
(1,1,4,2,3,6,'2026-09-06T20:00:00',N'Završena',0,0,2065,NULL),
(1,1,8,3,4,7,'2026-09-07T18:00:00',N'Završena',2,3,2238,NULL),
(1,1,9,4,1,8,'2026-09-08T19:00:00',N'Završena',4,2,2411,NULL),
(1,1,1,1,2,1,'2026-09-09T20:00:00',N'Zakazana',NULL,NULL,NULL,NULL),
(1,1,3,2,3,2,'2026-09-10T18:00:00',N'Zakazana',NULL,NULL,NULL,NULL),
(2,3,7,7,8,3,'2026-09-11T19:00:00',N'Zakazana',NULL,NULL,NULL,NULL),
(2,3,9,8,5,4,'2026-09-12T20:00:00',N'Zakazana',NULL,NULL,NULL,NULL),
(2,3,2,5,6,5,'2026-09-13T18:00:00',N'Zakazana',NULL,NULL,NULL,NULL),
(2,3,4,6,7,6,'2026-09-14T19:00:00',N'Zakazana',NULL,NULL,NULL,NULL),
(2,3,8,7,8,7,'2026-09-15T20:00:00',N'Zakazana',NULL,NULL,NULL,NULL),
(2,3,9,8,5,8,'2026-09-16T18:00:00',N'Zakazana',NULL,NULL,NULL,NULL),
(2,3,1,5,6,1,'2026-09-17T19:00:00',N'Zakazana',NULL,NULL,NULL,NULL),
(2,3,3,6,7,2,'2026-09-18T20:00:00',N'Zakazana',NULL,NULL,NULL,NULL),
(3,5,7,9,10,3,'2026-09-19T18:00:00',N'Zakazana',NULL,NULL,NULL,NULL),
(3,5,9,10,9,4,'2026-09-20T19:00:00',N'Zakazana',NULL,NULL,NULL,NULL),
(3,5,2,9,10,5,'2026-09-21T20:00:00',N'Zakazana',NULL,NULL,NULL,NULL),
(3,5,4,10,9,6,'2026-09-22T18:00:00',N'Zakazana',NULL,NULL,NULL,NULL),
(3,5,8,9,10,7,'2026-09-23T19:00:00',N'Zakazana',NULL,NULL,NULL,NULL),
(3,5,9,10,9,8,'2026-09-24T20:00:00',N'Zakazana',NULL,NULL,NULL,NULL);
```

Slika 15 - Unos podataka u tabelu Utakmice

2.4.14. Unos podataka u tabelu Nastupi

U tabelu Nastupi unose se nastupi sportista na utakmicama. Svaki nastup uključuje broj poena, broj asistencija i minute nastupa. Nastupi su raspoređeni po različitim utakmicama i sportistima, omogućavajući testiranje statističkih upita.

```
INSERT INTO sport.Nastupi(UtakmicaID,SportistaID,EkipaID,BrojPoena,BrojAsistencija,MinutaNastupa,Napomena) VALUES
(1,1,1,2.00,1,46,NULL),
(1,2,1,4.00,2,47,NULL),
(1,3,2,6.00,3,48,NULL),
(1,4,2,8.00,4,49,NULL),
(2,5,3,10.00,5,50,NULL),
(2,6,3,12.00,6,51,NULL),
(2,7,4,14.00,0,52,NULL),
(2,8,4,16.00,1,53,NULL),
(3,9,5,0.00,2,54,NULL),
(3,10,5,2.00,3,55,NULL),
(3,11,6,4.00,4,56,NULL),
(3,12,6,6.00,5,57,NULL),
(4,13,7,8.00,6,58,NULL),
(4,14,7,10.00,0,59,NULL),
(4,15,8,12.00,1,60,NULL),
```

Slika 16 - Unos podataka u tabelu Nastupi

2.4.15. Unos podataka u tabelu Kazne

U tabelu Kazne unose se kazne izrečene na utakmicama. Unose se žuti kartoni, crveni karton, timska kazna, tehnička greška i lična greška. Kazne mogu biti izrečene sportistima ili ekipama, sa odgovarajućim novčanim iznosima.

```
INSERT INTO sport.Kazne(UtakmicaID,SportistaID,EkipaID,VrstaKazne,MinutKazne,NovcaniIznos,Opis) VALUES
(1,1,1,N'Žuti karton',34,0,N'Nesportski prekršaj'),(2,3,2,N'Žuti karton',67,0,N'Prigovor sudiji'),
(3,5,3,N'Crveni karton',82,250.00,N'Grub prekršaj'),(4,NULL,4,N'Timska kazna',NULL,500.00,N'Kašnjenje na početak'),
(5,9,5,N'Tehnička greška',22,100.00,N'Nesportsko ponašanje'),(6,11,6,N'Lična greška',15,0,N'Peta lična greška');
```

Slika 17 - Unos podataka u tabelu Kazne

2.4.16. Unos podataka u tabelu Sponzori

U tabelu Sponzori unosi se šest sponzora. Unose se SportPlus, Balkan Telecom, Energy Drink Adria, Bank Europa, Auto Regional i Tech Solutions. Svaki sponzor ima svoj kontakt email i telefon.

```
INSERT INTO prodaja.Sponzori(Naziv,KontaktEmail,Telefon) VALUES
(N'SportPlus',N'kontakt@sportplus.example',N'051111111'),(N'Balkan Telecom',N'sponzorstva@balkantel.example',N'033222222'),
(N'Energy Drink Adria',N'marketing@energyadria.example',N'011333333'),(N'Bank Europa',N'promo@bankeuropa.example',N'01 444 444'),
(N'Auto Regional',N'info@autoregional.example',N'051555555'),(N'Tech Solutions',N'office@techsolutions.example',N'051666666');
```

Slika 18 - Unos podataka u tabelu Sponzori

2.4.17. Unos podataka u tabelu TakmicenjeSponzori

U tabelu TakmicenjeSponzori unose se veze između takmičenja i sponzora. Svaki unos uključuje iznos sponzorstva i datum ugovora. Ova tabela omogućava praćenje finansijskih aranžmana između takmičenja i sponzora.

```
INSERT INTO prodaja.TakmicenjeSponzori(TakmicenjeID,SponzorID,Iznos,DatumUgovora) VALUES
(1,1,40000,'2026-07-01'),(1,2,60000,'2026-07-03'),(2,2,45000,'2026-07-10'),(2,3,30000,'2026-07-11'),
(3,4,25000,'2026-08-01'),(4,5,18000,'2026-08-05'),(5,1,20000,'2026-04-01'),(5,6,35000,'2026-04-05'),(6,3,15000,'2026-08-10');
GO
```

Slika 19 - Unos podataka u tabelu TakmicenjeSponzori

2.4.18. Unos podataka u tabelu Ulaznice

U tabelu Ulaznice unose se ulaznice za različite utakmice. Unosi se šezdeset ulaznica, od kojih je četrdeset pet prodatih, a petnaest slobodnih. Svaka ulaznica ima svoj sektor, red, broj sjedišta i cijenu. Prodane ulaznice imaju datum prodaje i email kupca.

```
INSERT INTO prodaja.Ulaznice(UtakmicaID,Sektor,RedBroj,SjedisteBroj,Cijena,DatumProdaje,StatusUlaznice,KupacEmail) VALUES
(1,'B','2',2,15.00,'2026-08-02T11:00:00','N'Prodato','N'kupac01@example.com'),
(2,'C','3',3,20.00,'2026-08-03T12:00:00','N'Prodato','N'kupac02@example.com'),
(3,'A','4',4,25.00,'2026-08-04T13:00:00','N'Prodato','N'kupac03@example.com'),
(4,'B','5',5,10.00,'2026-08-05T14:00:00','N'Prodato','N'kupac04@example.com'),
(5,'C','6',6,15.00,'2026-08-06T15:00:00','N'Prodato','N'kupac05@example.com'),
(6,'A','7',7,20.00,'2026-08-07T16:00:00','N'Prodato','N'kupac06@example.com'),
(7,'B','8',8,25.00,'2026-08-08T17:00:00','N'Prodato','N'kupac07@example.com'),
(8,'C','9',9,10.00,'2026-08-09T18:00:00','N'Prodato','N'kupac08@example.com'),
(9,'A','10',10,15.00,'2026-08-10T19:00:00','N'Prodato','N'kupac09@example.com'),
(10,'B','1',11,20.00,'2026-08-11T10:00:00','N'Prodato','N'kupac10@example.com'),
```

Slika 20 - Unos podataka u tabelu Ulaznice

2.4.19. Unos podataka u tabelu Placanja

U tabelu Placanja unose se uplate za prodane ulaznice. Unosi se četrdeset pet uplata sa različitim načinima plaćanja, uključujući gotovinu, karticu, online i vaucher. Sve uplate su označene kao plaćene, sa odgovarajućim datumima. Za online i kartične transakcije, evidentiran je i broj transakcije.

```
INSERT INTO prodaja.Placanja(UlaznicaID,DatumPlacanja,Iznos,NacinPlacanja,StatusPlacanja,BrojTransakcije) VALUES
(1,'2026-08-02T12:00:00',15.00,'N'Kartica','N'Placeno','N'TXN-00001'),
(2,'2026-08-03T13:00:00',20.00,'N'Online','N'Placeno','N'TXN-00002'),
(3,'2026-08-04T14:00:00',25.00,'N'Vaucer','N'Placeno','N'TXN-00003'),
(4,'2026-08-05T15:00:00',10.00,'N'Gotovina','N'Placeno',NULL),
(5,'2026-08-06T16:00:00',15.00,'N'Kartica','N'Placeno','N'TXN-00005'),
(6,'2026-08-07T17:00:00',20.00,'N'Online','N'Placeno','N'TXN-00006'),
(7,'2026-08-08T18:00:00',25.00,'N'Vaucer','N'Placeno','N'TXN-00007'),
(8,'2026-08-09T19:00:00',10.00,'N'Gotovina','N'Placeno',NULL),
(9,'2026-08-10T11:00:00',15.00,'N'Kartica','N'Placeno','N'TXN-00009'),
(10,'2026-08-11T12:00:00',20.00,'N'Online','N'Placeno','N'TXN-00010'),
(11,'2026-08-12T13:00:00',25.00,'N'Vaucer','N'Placeno','N'TXN-00011'),
(12,'2026-08-13T14:00:00',10.00,'N'Gotovina','N'Placeno',NULL),
(13,'2026-08-14T15:00:00',15.00,'N'Kartica','N'Placeno','N'TXN-00013'),
(14,'2026-08-15T16:00:00',20.00,'N'Online','N'Placeno','N'TXN-00014'),
(15,'2026-08-16T17:00:00',25.00,'N'Vaucer','N'Placeno','N'TXN-00015'),
(16,'2026-08-17T18:00:00',10.00,'N'Gotovina','N'Placeno',NULL),
```

Slika 21 - Unos podataka u tabelu Placanja

2.5. Kreiranje pogleda

Pogledi su virtualne tabele koje ne čuvaju podatke fizički, već predstavljaju rezultat unaprijed definisanog upita. Oni pojednostavljuju pristup podacima tako što omogućavaju korisnicima da

rade sa jednostavnijom strukturom, skrivajući kompleksnost osnovnih tabela i relacija. Također, pogledi povećavaju sigurnost jer mogu prikazati samo određene kolone ili redove, skrivajući osjetljive podatke.

2.5.1. Pogled vw_RasporedUtakmica

Prvi pogled, nazvan vw_RasporedUtakmica, kreiran je da pruži sveobuhvatan pregled rasporeda utakmica sa svim relevantnim detaljima. Ovaj pogled spaja podatke iz tabela Utakmice, Takmicenja, Faze, Ekipe, Objekti, Gradovi i Sudije, prikazujući informacije koje su korisne za operativni rad i prikaz rasporeda.

```
CREATE VIEW izvjestaji.vw_RasporedUtakmica AS
SELECT u.UtakmicaID,t.Naziv AS Takmicenje,f.Naziv AS Faza,u.DatumVrijeme,
       ISNULL(ed.Naziv,N'Individualni nastup') AS Domacin,
       ISNULL(eg.Naziv,N'Individualni nastup') AS Gost,
       o.Naziv AS Objekat,g.Naziv AS Grad,
       s.Ime+N' '+s.Prezime AS Sudija,u.StatusUtakmice,
       CASE WHEN u.RezultatDomaci IS NULL THEN N'-'
            ELSE CAST(u.RezultatDomaci AS NVARCHAR(10))+N':'+CAST(u.RezultatGosti AS
NVARCHAR(10)) END AS Rezultat
FROM sport.Utakmice u
INNER JOIN sport.Takmicenja t ON u.TakmicenjeID=t.TakmicenjeID
LEFT JOIN sport.Faze f ON u.FazaID=f.FazaID
LEFT JOIN sport.Ekipe ed ON u.DomacaEkipaID=ed.EkipaID
LEFT JOIN sport.Ekipe eg ON u.GostujucaEkipaID=eg.EkipaID
INNER JOIN sport.Objekti o ON u.ObjekatID=o.ObjekatID
INNER JOIN sif.Gradovi g ON o.GradID=g.GradID
LEFT JOIN sport.Sudije s ON u.SudijaID=s.SudijaID;
```

Pogled prikazuje identifikator utakmice, naziv takmičenja, fazu, datum i vrijeme utakmice, domaćina, gosta, objekat, grad, sudiju, status utakmice i rezultat. Kreiranjem ovog pogleda, korisnici ne moraju svaki put pisati kompleksne JOIN upite da bi dobili potrebne informacije.

2.5.2. Pogled vw_ProdajaPoUtakmici

Drugi pogled, nazvan vw_ProdajaPoUtakmici, kreiran je za potrebe finansijskog izvještavanja. Ovaj pogled prikazuje prodaju ulaznica po utakmicama, uključujući broj kreiranih ulaznica, broj prodatih ulaznica i ukupan prihod.

```
CREATE VIEW izvjestaji.vw_ProdajaPoUtakmici AS
SELECT u.UtakmicaID,t.Naziv AS Takmicenje,u.DatumVrijeme,
       COUNT(ul.UlaznicaID) AS KreiranoUlaznica,
       SUM(CASE WHEN ul.StatusUlaznice=N'Prodata' THEN 1 ELSE 0 END) AS
ProdatoUlaznica,
       ISNULL(SUM(CASE WHEN p.StatusPlacanja=N'Placeno' THEN p.Iznos ELSE 0 END),0)
AS Prihod
FROM sport.Utakmice u
INNER JOIN sport.Takmicenja t ON u.TakmicenjeID=t.TakmicenjeID
LEFT JOIN prodaja.Ulaznice ul ON u.UtakmicaID=ul.UtakmicaID
LEFT JOIN prodaja.Placanja p ON ul.UlaznicaID=p.UlaznicaID
GROUP BY u.UtakmicaID,t.Naziv,u.DatumVrijeme;
```

Pogled prikazuje identifikator utakmice, naziv takmičenja, datum i vrijeme utakmice, broj kreiranih ulaznica, broj prodatih ulaznica i prihod. Korištenje funkcije ISNULL osigurava da utakmice bez prodaje budu prikazane sa nulnim prihodom. Filtriranje na StatusPlacanja jednako Placeno osigurava da se u obzir uzimaju samo uspješne uplate.

2.5.3. Pogled vw_StatistikaSportista

Treći pogled, nazvan vw_StatistikaSportista, kreiran je za potrebe statističkog izvještavanja o sportistima. Ovaj pogled prikazuje broj nastupa, ukupno poena, prosjek poena i ukupno asistencija za svakog sportistu.

```
CREATE VIEW izvjestaji.vw_StatistikaSportista AS
SELECT sp.SportistaID,sp.Ime,sp.Prezime,COUNT(n.NastupID) AS BrojNastupa,
```

```

        ISNULL(SUM(n.BrojPoena),0) AS UkupnoPoena,
        ISNULL(AVG(n.BrojPoena),0) AS ProsjekPoena,
        ISNULL(SUM(n.BrojAsistencija),0) AS UkupnoAsistencija
FROM sport.Sportisti sp LEFT JOIN sport.Nastupi n ON sp.SportistaID=n.SportistaID
GROUP BY sp.SportistaID,sp.Ime,sp.Prezime;

```

Pogled prikazuje identifikator sportiste, ime, prezime, broj nastupa, ukupno poena, prosjek poena i ukupno asistencija. Korištenje LEFT JOIN-a osigurava da se prikažu i sportisti koji još nisu imali nijedan nastup, sa nultim vrijednostima za statistiku.

2.6. Kreiranje korisničkih funkcija

Korisnički definisane funkcije su objekti koji primaju parametre, izvršavaju određenu logiku i vraćaju vrijednost. Za razliku od procedura, funkcije se mogu koristiti unutar SELECT upita, što ih čini vrlo fleksibilnim za ponovno korištenje koda.

2.6.1. Funkcija fn_Starost

Prva funkcija, nazvana fn_Starost, izračunava tačnu starost osobe na osnovu datuma rođenja. Ova funkcija prima datum rođenja kao parametar, a vraća cjelobrojnu vrijednost koja predstavlja starost u godinama. Logika funkcije koristi DATEDIFF za izračunavanje razlike u godinama, a zatim provjerava da li je datum rođendana u tekućoj godini već prošao, kako bi se dobila tačna starost.

```

CREATE FUNCTION sport.fn_Starost(@DatumRodjenja DATE)
RETURNS INT
AS
BEGIN
    DECLARE @Starost INT;
    SET @Starost=DATEDIFF(YEAR,@DatumRodjenja,GETDATE())-
    CASE WHEN
    DATEADD(YEAR,DATEDIFF(YEAR,@DatumRodjenja,GETDATE()),@DatumRodjenja)>G
ETDATE() THEN 1 ELSE 0 END;

```



```
RETURN @Starost;  
END;
```

Ova funkcija je implementirana kao skalarna funkcija, što znači da vraća jednu vrijednost. Može se koristiti unutar SELECT upita za izračunavanje starosti sportista prilikom prikaza statistike.

2.6.2. Funkcija fn_PrihodUtakmice

Druga funkcija, nazvana fn_PrihodUtakmice, izračunava ukupan prihod ostvaren od jedne utakmice kroz prodaju ulaznica. Ova funkcija prima identifikator utakmice kao parametar, a vraća ukupan iznos plaćenih ulaznica za tu utakmicu. Funkcija sumira sve uplate koje su povezane sa ulaznicama za datu utakmicu, pri čemu se uzimaju u obzir samo uplate sa statusom Placeno.

```
CREATE FUNCTION prodaja.fn_PrihodUtakmice(@UtakmicaID INT)  
RETURNS DECIMAL(14,2)  
AS  
BEGIN  
    DECLARE @Prihod DECIMAL(14,2);  
    SELECT @Prihod=ISNULL(SUM(p.Iznos),0)  
    FROM prodaja.Ulaznice u INNER JOIN prodaja.Placanja p ON u.UlaznicaID=p.UlaznicaID  
    WHERE u.UtakmicaID=@UtakmicaID AND p.StatusPlacanja=N'Placeno';  
    RETURN ISNULL(@Prihod,0);  
END;
```

2.6.3. Funkcija fn_UtakmiceEkipe

Treća funkcija, nazvana fn_UtakmiceEkipe, prikazuje sve utakmice određene ekipe. Ova funkcija prima identifikator ekipe kao parametar, a vraća tabelu sa svim utakmicama gdje je data ekipa bila domaćin ili gost. Ova funkcija je implementirana kao tabelarna funkcija, što znači da vraća skup redova.

```
CREATE FUNCTION sport.fn_UtakmiceEkipe(@EkipaID INT)
```

RETURNS TABLE

AS RETURN(

```
SELECT u.UtakmicaID,u.DatumVrijeme,u.StatusUtakmice,  
       d.Naziv AS Domacin,g.Naziv AS Gost,u.RezultatDomaci,u.RezultatGosti  
FROM sport.Utakmice u  
LEFT JOIN sport.Ekipe d ON u.DomacaEkipaID=d.EkipaID  
LEFT JOIN sport.Ekipe g ON u.GostujucaEkipaID=g.EkipaID  
WHERE u.DomacaEkipaID=@EkipaID OR u.GostujucaEkipaID=@EkipaID  
);
```

Implementacija koristi LEFT JOIN za povezivanje sa ekipama kao domaćinima i gostima, a WHERE uslov filtrira utakmice gdje se data ekipa pojavljuje u bilo kojoj ulozi.

2.7. Kreiranje uskladištenih procedura radi automatizacije

Uskladištene procedure su prethodno kompajlirani SQL kod koji se izvršava na serveru. One omogućavaju ponovno korištenje koda, bolje performanse jer se SQL kod kompajlira jednom prilikom kreiranja procedure, a zatim se izvršava više puta bez ponovnog kompajliranja, te povećanu sigurnost jer se korisnicima može dati dozvola za izvršavanje procedure bez davanja direktnog pristupa tabelama.

2.7.1. Procedura sp_EvidentirajRezultat

Prva procedura, nazvana sp_EvidentirajRezultat, automatizuje proces evidentiranja rezultata utakmice. Ova procedura prihvata parametre koji definišu rezultat, provjerava ispravnost podataka i ažurira utakmicu sa rezultatom i statusom. Cijela operacija je obavijena transakcijom, što znači da će se ili sve promjene primijeniti ili nijedna, u slučaju greške.

```
CREATE PROCEDURE sport.sp_EvidentirajRezultat  
    @UtakmicaID INT,@Domaci INT,@Gosti INT,@BrojGledalaca INT=NULL  
AS  
BEGIN
```

```

SET NOCOUNT ON; SET XACT_ABORT ON;
BEGIN TRY
    BEGIN TRANSACTION;
    IF NOT EXISTS(SELECT 1 FROM sport.Utakmice WITH(UPDLOCK,HOLDLOCK)
WHERE UtakmicaID=@UtakmicaID)
        RAISERROR(N'Utakmica ne postoji.',16,1);
    IF @Domaci<0 OR @Gosti<0 RAISERROR(N'Rezultat ne može biti negativan.',16,1);
    IF @BrojGledalaca IS NOT NULL AND @BrojGledalaca<0 RAISERROR(N'Broj gledalaca
nije ispravan.',16,1);
    UPDATE sport.Utakmice SET RezultatDomaci=@Domaci,RezultatGosti=@Gosti,
        BrojGledalaca=@BrojGledalaca,StatusUtakmice=N'Završena'
    WHERE UtakmicaID=@UtakmicaID;
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF XACT_STATE()<>0 ROLLBACK TRANSACTION;
    DECLARE @P NVARCHAR(4000); SET @P=ERROR_MESSAGE();
    RAISERROR(@P,16,1);
END CATCH
END;

```

Procedura prihvata sljedeće parametre: UtakmicaID, Domaci, Gosti i BrojGledalaca. Unutar procedure, prvo se provjerava da li utakmica postoji u bazi, koristeći UPDLOCK i HOLDLOCK hintove za zaključavanje reda tokom transakcije. Zatim se provjerava da li su rezultati nenegativni i da li je broj gledalaca, ako je naveden, ispravan. Nakon što su provjere prošle, procedura ažurira utakmicu sa rezultatima i postavlja status na Završena. Ako sve naredbe izvrše uspješno, transakcija se potvrđuje. U slučaju bilo kakve greške, transakcija se poništava i greška se prosleđuje pozivaocu procedure.

2.7.2. Procedura sp_ProdajUlaznicu

Druga procedura, nazvana sp_ProdajUlaznicu, automatizuje proces prodaje ulaznice. Ova procedura prihvata identifikator ulaznice, email kupca i način plaćanja, provjerava dostupnost ulaznice, ažurira njen status i kreira uplatu.

```
CREATE PROCEDURE prodaja.sp_ProdajUlaznicu
    @UlaznicaID INT,@KupacEmail NVARCHAR(100),@NacinPlacanja NVARCHAR(20)
AS
BEGIN
    SET NOCOUNT ON; SET XACT_ABORT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        DECLARE @Cijena DECIMAL(10,2);
        SELECT @Cijena=Cijena FROM prodaja.Ulaznice WITH(UPDLOCK,HOLDLOCK)
        WHERE UlaznicaID=@UlaznicaID AND StatusUlaznice=N'Slobodna';
        IF @Cijena IS NULL RAISERROR(N'Ulaznica ne postoji ili nije slobodna.',16,1);
        IF @NacinPlacanja NOT IN(N'Gotovina',N'Kartica',N'Online',N'Vaucer')
        RAISERROR(N'Nepodržan način plaćanja.',16,1);
        UPDATE prodaja.Ulaznice SET
        StatusUlaznice=N'Prodata',DatumProdaje=GETDATE(),KupacEmail=@KupacEmail
        WHERE UlaznicaID=@UlaznicaID;
        INSERT INTO
        prodaja.Placanja(UlaznicaID,Iznos,NacinPlacanja,StatusPlacanja,BrojTransakcije)
        VALUES(@UlaznicaID,@Cijena,@NacinPlacanja,N'Placeno',
            CASE WHEN @NacinPlacanja=N'Gotovina' THEN NULL ELSE N'TXN-
'+REPLACE(CONVERT(NVARCHAR(36),NEWID()),N'-'',N'') END);
        SELECT @UlaznicaID AS ProdataUlaznicaID,@Cijena AS PlaceniIznos;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF XACT_STATE()<>0 ROLLBACK TRANSACTION;
```

```

DECLARE @P NVARCHAR(4000); SET @P=ERROR_MESSAGE();
RAISERROR(@P,16,1);
END CATCH
END;

```

Procedura prvo dohvata cijenu ulaznice uz zaključavanje reda kako bi se spriječilo duplo prodavanje iste ulaznice. Zatim provjerava da li je ulaznica slobodna i da li je način plaćanja podržan. Nakon toga, ažurira status ulaznice na Prodata, postavlja datum prodaje i email kupca. Zatim unosi uplatu u tabelu Placanja sa odgovarajućim podacima. Za načine plaćanja koji nisu gotovina, generiše se jedinstveni broj transakcije korištenjem NEWID() funkcije. Na kraju, procedura vraća identifikator prodate ulaznice i plaćeni iznos.

2.7.3. Procedura sp_RasporedZaPeriod

Treća procedura, nazvana sp_RasporedZaPeriod, kreirana je za potrebe izvještavanja o rasporedu utakmica u određenom vremenskom periodu. Ova procedura prihvata početni i krajnji datum kao parametre i vraća sve utakmice koje se nalaze u tom periodu, koristeći pogled vw_RasporedUtakmica.

```

CREATE PROCEDURE izvjestaji.sp_RasporedZaPeriod @DatumOd DATETIME,@DatumDo
DATETIME
AS
BEGIN
SET NOCOUNT ON;
SELECT * FROM izvjestaji.vw_RasporedUtakmica
WHERE DatumVrijeme>=@DatumOd AND DatumVrijeme<@DatumDo ORDER BY
DatumVrijeme;
END;

```

Ova procedura je korisna za operativni rad jer omogućava brz pregled rasporeda za bilo koji vremenski interval.

2.7.4. Procedura sp_PrihodPoTakmicenju

Četvrta procedura, nazvana sp_PrihodPoTakmicenju, kreirana je za potrebe finansijskog izvještavanja o prihodima po takmičenjima u određenom vremenskom periodu. Ova procedura prihvata početni i krajnji datum kao parametre i vraća prihod po takmičenjima za taj period.

```
CREATE PROCEDURE izvjestaji.sp_PrihodPoTakmicenju @DatumOd
DATETIME,@DatumDo DATETIME
AS
BEGIN
SET NOCOUNT ON;
SELECT t.Naziv,COUNT(DISTINCT u.UtakmicaID) AS BrojUtakmica,
COUNT(DISTINCT ul.UlaznicaID) AS BrojProdatihUlaznica,
ISNULL(SUM(p.Iznos),0) AS Prihod
FROM sport.Takmicenja t
LEFT JOIN sport.Utakmice u ON t.TakmicenjeID=u.TakmicenjeID
LEFT JOIN prodaja.Ulaznice ul ON u.UtakmicaID=ul.UtakmicaID AND
ul.StatusUlaznice=N'Prodata'
LEFT JOIN prodaja.Placanja p ON ul.UlaznicaID=p.UlaznicaID AND
p.StatusPlacanja=N'Placeno'
AND p.DatumPlacanja>=@DatumOd AND p.DatumPlacanja<@DatumDo
GROUP BY t.Naziv ORDER BY Prihod DESC;
END;
```

Procedura spaja podatke iz tabela Takmicenja, Utakmice, Ulaznice i Placanja, grupira ih po takmičenjima i prikazuje broj utakmica, broj prodatih ulaznica i ukupan prihod. Rezultati su sortirani po prihodu opadajuće, što omogućava brz uvid u najprofitabilnija takmičenja.

2.8. KREIRANJE TRIGERA ZA AUTOMATSKO IZVRŠAVANJE RADNJI

Trigeri su posebne vrste uskladištenih procedura koje se ne pozivaju eksplicitno, već se automatski izvršavaju kada se desi određeni događaj na tabeli, kao što su INSERT, UPDATE ili DELETE

operacije. Trigeri se koriste za održavanje integriteta podataka, automatsko ažuriranje povezanih podataka, logiranje promjena i implementaciju složenih poslovnih pravila.

2.8.1. Triger trg_Utakmice_Dnevnik

Prvi trigger, nazvan trg_Utakmice_Dnevnik, aktivira se nakon INSERT, UPDATE ili DELETE operacije nad tabelom Utakmice. Njegova svrha je da evidentira svaku promjenu nad utakmicama u dnevnik, omogućavajući kasniju reviziju i praćenje istorije.

```
CREATE TRIGGER sport.trg_Utakmice_Dnevnik ON sport.Utakmice
AFTER INSERT,UPDATE,DELETE AS
BEGIN
    SET NOCOUNT ON;
    INSERT INTO
audit.UtakmiceDnevnik(UtakmicaID,Akcija,StariStatus,NoviStatus,StariRezultat,NoviRezultat)
    SELECT ISNULL(i.UtakmicaID,d.UtakmicaID),
    CASE WHEN d.UtakmicaID IS NULL THEN N'INSERT' WHEN i.UtakmicaID IS NULL
    THEN N'DELETE' ELSE N'UPDATE' END,
    d.StatusUtakmice,i.StatusUtakmice,
    CASE WHEN d.RezultatDomaci IS NULL THEN NULL ELSE CAST(d.RezultatDomaci AS
NVARCHAR(10))+N':' +CAST(d.RezultatGosti AS NVARCHAR(10)) END,
    CASE WHEN i.RezultatDomaci IS NULL THEN NULL ELSE CAST(i.RezultatDomaci AS
NVARCHAR(10))+N':' +CAST(i.RezultatGosti AS NVARCHAR(10)) END
    FROM inserted i FULL OUTER JOIN deleted d ON i.UtakmicaID=d.UtakmicaID;
END;
```

Trigger koristi specijalne tabele inserted i deleted koje SQL Server automatski kreira u kontekstu triggera. Tabela inserted sadrži nove vrijednosti nakon INSERT ili UPDATE operacije, dok tabela deleted sadrži stare vrijednosti prije DELETE ili UPDATE operacije. Trigger pomoću FULL OUTER JOIN-a kombinuje ove dvije tabele kako bi odredio vrstu akcije. Ako je inserted NULL, radi se o DELETE operaciji. Ako je deleted NULL, radi se o INSERT operaciji. Ako obje tabele imaju vrijednosti, radi se o UPDATE operaciji. Za svaku promjenu, trigger upisuje zapis u dnevnik

sa odgovarajućim podacima, uključujući stari i novi status, stari i novi rezultat, te korisnika koji je izvršio promjenu.

2.8.2. Trigger trg_Ulaznice_CijenaDnevnik

Drugi trigger, nazvan trg_Ulaznice_CijenaDnevnik, aktivira se nakon UPDATE operacije nad tabelom Ulaznice. Njegova svrha je da evidentira svaku promjenu cijene ulaznice u log tabelu. Trigger prvo provjerava da li je UPDATE operacija uticala na kolonu Cijena. Zatim upisuje zapis u log tabelu sa starom cijenom i novom cijenom za svaku ulaznicu čija je cijena promijenjena.

```
CREATE TRIGGER prodaja.trg_Ulaznice_CijenaDnevnik ON prodaja.Ulaznice
AFTER UPDATE AS
BEGIN
SET NOCOUNT ON;
INSERT INTO audit.CijeneUlaznicaDnevnik(UlaznicaID,StaraCijena,NovaCijena)
SELECT i.UlaznicaID,d.Cijena,i.Cijena FROM inserted i INNER JOIN deleted d ON
i.UlaznicaID=d.UlaznicaID
WHERE i.Cijena<>d.Cijena;
END;
```

2.8.3. Trigger trg_Utakmice_Kapacitet

Treći trigger, nazvan trg_Utakmice_Kapacitet, aktivira se nakon INSERT ili UPDATE operacije nad tabelom Utakmice. Njegova svrha je da provjeri da li broj gledalaca prelazi kapacitet objekta na kojem se utakmica održava.

```
CREATE TRIGGER sport.trg_Utakmice_Kapacitet ON sport.Utakmice
AFTER INSERT,UPDATE AS
BEGIN
SET NOCOUNT ON;
IF EXISTS(SELECT 1 FROM inserted i INNER JOIN sport.Objekti o ON
i.ObjekatID=o.ObjekatID
```



```
WHERE i.BrojGledalaca IS NOT NULL AND i.BrojGledalaca>o.Kapacitet)  
BEGIN RAISERROR(N'Broj gledalaca prelazi kapacitet objekta.',16,1); ROLLBACK  
TRANSACTION; RETURN; END;  
END;
```

Trigger spaja podatke iz inserted sa tabelom Objekti kako bi provjerio da li je broj gledalaca, ako je naveden, veći od kapaciteta objekta. Ako jeste, trigger generiše grešku i poništava transakciju, čime se sprečava unos neispravnih podataka.

2.9. Primjeri DDL i DML naredbi u bazi podataka

DDL naredbe se koriste za definisanje i izmjenu strukture baze podataka, dok DML naredbe služe za manipulaciju podacima. U ovom poglavlju prikazani su konkretni primjeri obje vrste naredbi nad bazom SportskaTakmicenja.

2.9.1. ALTER naredba za izmjenu strukture

ALTER naredba se koristi za izmjenu postojećih objekata u bazi podataka. U nastavku je prikazan primjer dodavanja nove kolone u tabelu Sportisti. Ova kolona će čuvati informaciju o tome da li sportista želi da prima obavijesti.

```
ALTER TABLE sport.Sportisti ADD PrimaObavijesti BIT NOT NULL CONSTRAINT  
DF_Sportisti_PrimaObavijesti DEFAULT 1
```

2.9.2. INSERT naredba za unos podataka

INSERT naredba se koristi za unos novih podataka u tabele. U nastavku je prikazan primjer unosa testnog sportiste u tabelu Sportisti. Ova naredba se koristi za registraciju novih sportista u sistem.

```
INSERT INTO sport.Sportisti(DrzavaID,Ime,Prezime,DatumRodjenja,Pol,Email)  
VALUES(1,N'Test',N'Sportista','2000-01-01',N'M',N'test.sportista@example.com')
```

2.9.3. UPDATE naredba za izmjenu podataka

UPDATE naredba se koristi za izmjenu postojećih podataka u tabelama. U nastavku je prikazan primjer ažuriranja kolone PrimaObavijesti za testnog sportistu.

```
UPDATE sport.Sportisti SET PrimaObavijesti=0 WHERE  
Email=N'test.sportista@example.com'
```

2.9.4. DELETE naredba za brisanje podataka

DELETE naredba se koristi za brisanje redova iz tabela. U nastavku je prikazan primjer brisanja testnog sportiste. Ova naredba je obavijena transakcijom koja se poništava, kako bi se testni podaci mogli vratiti.

```
BEGIN TRANSACTION;  
DELETE FROM sport.Sportisti WHERE Email=N'test.sportista@example.com';  
ROLLBACK TRANSACTION;
```

2.9.5. TRUNCATE naredba za brzo brisanje svih podataka

TRUNCATE je DDL naredba koja briše sve redove iz tabele, ali zadržava strukturu tabele. Brža je od DELETE jer ne zapisuje svaki red pojedinačno. Primjer prikazuje brisanje svih zapisa iz tabele DnevniPrihod.

```
TRUNCATE TABLE izvjestaji.DnevniPrihod
```

Razlika između DELETE i TRUNCATE je značajna. DELETE je sporija jer zapisuje svaki red i može koristiti WHERE uslov, aktivira trigere i ne resetuje IDENTITY. TRUNCATE je veoma brza, ne može koristiti WHERE uslov, ne aktivira trigere i resetuje IDENTITY na početnu vrijednost.

2.10. Primjeri korištenja različitih funkcija u bazi podataka

SQL Server nudi bogat skup ugrađenih funkcija koje se mogu koristiti u upitima. U ovom poglavlju prikazani su primjeri korištenja različitih tipova funkcija nad bazom SportskaTakmicenja.

2.10.1. Datumske funkcije

Datumske funkcije se koriste za rad sa datumima i vremenom. U primjeru ispod, za svakog sportistu se prikazuje datum registracije i broj dana od registracije do danas, korištenjem DATEDIFF funkcije sa DAY parametrom. Također, koristi se korisnička funkcija fn_Starost za izračunavanje starosti sportiste.

```
SELECT SportistaID, Ime, Prezime, sport.fn_Starost(DatumRodjenja) AS Starost,  
DATEDIFF(DAY, DatumRegistracije, GETDATE()) AS DanaOdRegistracije FROM  
sport.Sportisti
```

2.10.2. String funkcije

String funkcije se koriste za manipulaciju tekstualnim podacima. U primjeru ispod, prikazana je konverzija imena u velika slova, prezimena u mala slova, izračunavanje dužine kombinacije imena i prezimena, te izdvajanje prva tri slova prezimena. Također, koristi se ISNULL za zamjenu NULL email adresa sa tekстом Nema email.

```
SELECT SportistaID, UPPER(Ime) AS ImeVelikim, LOWER(Prezime) AS PrezimeMalim,  
LEN(Ime+Prezime) AS BrojZnakova, LEFT(Prezime,3) AS PrvaTri, ISNULL>Email,N'Nema  
email') AS EmailPrikaz FROM sport.Sportisti
```

2.10.3. Agregatne funkcije

Agregatne funkcije se koriste za izračunavanje statističkih vrijednosti nad skupovima podataka. U primjeru ispod, izračunava se ukupan broj sportista, najstariji datum rođenja, najmlađi datum rođenja i prosječna visina sportista.

```
SELECT COUNT(*) AS BrojSportista, MIN(DatumRodjenja) AS NajstarijiDatum,  
MAX(DatumRodjenja) AS NajmladjiDatum, AVG(VisinaCm) AS ProsjecnaVisina FROM  
sport.Sportisti
```

2.10.4. Numeričke funkcije

Numeričke funkcije se koriste za matematičke operacije nad numeričkim podacima. U primjeru ispod, prikazane su različite numeričke operacije nad cijenama ulaznica, uključujući cijenu sa porezom, zaokruživanje i kategorizaciju cijena.

```
SELECT UlaznicaID, Cijena, ROUND(Cijena*1.17,2) AS CijenaSaPDV, CEILING(Cijena) AS  
Navise, FLOOR(Cijena) AS Nanize, CASE WHEN Cijena<15 THEN N'Povoljna' WHEN  
Cijena<=20 THEN N'Srednja' ELSE N'Premium' END AS Kategorija FROM prodaja.Ulaznice
```

2.11. PRIMJERI JOIN UPITA ZA KOMBINOVANJE PODATAKA IZ VIŠE TABELA

JOIN operacije se koriste za kombinovanje podataka iz dvije ili više tabela na osnovu veze između njih. U relacionim bazama podataka, podaci su često raspoređeni po različitim tabelama, a JOIN operacije omogućavaju njihovo povezivanje u smislene cijeline.

2.11.1. INNER JOIN

INNER JOIN vraća samo one redove koji imaju podudaranje u obe tabele koje se spajaju. Ovo je najčešće korištena vrsta JOIN-a. U primjeru ispod, prikazane su sve utakmice sa pripadajućim takmičenjima.

```
SELECT u.UtakmicaID, t.Naziv, u.DatumVrijeme FROM sport.Utakmice u INNER JOIN  
sport.Takmicenja t ON u.TakmicenjeID=t.TakmicenjeID
```

2.11.2. LEFT JOIN

LEFT JOIN vraća sve redove iz lijeve tabele, a samo podudarajuće iz desne tabele. Ako nema podudaranja, kolone iz desne tabele će biti NULL. U primjeru ispod, prikazani su svi sportisti, bez obzira da li imaju nastupe ili ne.

```
SELECT s.SportistaID, s.Ime, s.Prezime, COUNT(n.NastupID) AS BrojNastupa FROM  
sport.Sportisti s LEFT JOIN sport.Nastupi n ON s.SportistaID=n.SportistaID GROUP BY  
s.SportistaID, s.Ime, s.Prezime
```

2.11.3. RIGHT JOIN

RIGHT JOIN vraća sve redove iz desne tabele, a samo podudarajuće iz lijeve tabele. Ovo je simetrično LEFT JOIN-u. U primjeru ispod, prikazane su sve utakmice sa pripadajućim ulaznicama.

```
SELECT u.UtakmicaID, ul.UlaznicaID, ul.StatusUlaznice FROM prodaja.Ulaznice ul RIGHT  
JOIN sport.Utakmice u ON ul.UtakmicaID=u.UtakmicaID
```

2.11.4. FULL OUTER JOIN

FULL OUTER JOIN vraća sve redove iz obje tabele, spajajući gdje postoji podudaranje. Tamo gdje nema podudaranja, kolone iz suprotne tabele će biti NULL. U primjeru ispod, prikazane su sve ekipe i nastupi.

```
SELECT e.Naziv, n.NastupID, n.BrojPoena FROM sport.Ekipe e FULL OUTER JOIN  
sport.Nastupi n ON e.EkipaID=n.EkipaID
```

2.11.5. CROSS JOIN

CROSS JOIN vraća kartezijev proizvod dvije tabele, što znači da se svaki red iz prve tabele spaja sa svakim redom iz druge tabele. Ovo se rijetko koristi u praksi jer može proizvesti ogroman broj redova. U primjeru ispod, prikazane su sve moguće kombinacije sportova i sezona.

```
SELECT s.Naziv AS Sport, se.Naziv AS Sezona FROM sif.Sportovi s CROSS JOIN  
sport.Sezone se
```

2.11.6. SELF JOIN

SELF JOIN je spajanje tabele same sa sobom. Koristi se kada treba uporediti redove unutar iste tabele. U primjeru ispod, prikazani su parovi ekipa koje se takmiče u istom sportu.

```
SELECT e1.Naziv AS Ekipa1, e2.Naziv AS Ekipa2, e1.SportID FROM sport.Ekipe e1 INNER  
JOIN sport.Ekipe e2 ON e1.SportID=e2.SportID AND e1.EkipaID<e2.EkipaID
```

2.12. Primjeri podupita i skupovnih operatora

Podupiti su upiti ugniježđeni unutar drugih upita, dok skupovni operatori omogućavaju kombinovanje rezultata više upita.

2.12.1. Podupiti

Prvi primjer podupita prikazuje sportiste koji imaju iznadprosječan broj poena. Podupit izračunava prosječan broj poena po sportisti, a spoljašnji upit filtrira sportiste koji su iznad tog prosjeka.

```
SELECT s.Ime, s.Prezime, SUM(n.BrojPoena) AS Poeni FROM sport.Sportisti s INNER JOIN  
sport.Nastupi n ON s.SportistaID=n.SportistaID GROUP BY s.Ime, s.Prezime HAVING  
SUM(n.BrojPoena)>(SELECT AVG(x.Ukupno) FROM (SELECT SUM(BrojPoena) AS Ukupno  
FROM sport.Nastupi GROUP BY SportistaID)x)
```

Drugi primjer podupita prikazuje utakmice koje nemaju prodane ulaznice. Koristi se NOT EXISTS operator koji provjerava da li postoji prodana ulaznica za datu utakmicu.

```
SELECT u.UtakmicaID, u.DatumVrijeme FROM sport.Utakmice u WHERE NOT  
EXISTS(SELECT 1 FROM prodaja.Ulaznice ul WHERE ul.UtakmicaID=u.UtakmicaID AND  
ul.StatusUlaznice=N'Prodana')
```

Treći primjer podupita prikazuje takmičenja sa nagradnim fondom većim od prosjeka.

```
SELECT Naziv, NagradniFond FROM sport.Takmicenja WHERE NagradniFond>(SELECT  
AVG(NagradniFond) FROM sport.Takmicenja)
```

Četvrti primjer je korelisani podupit koji za svaku ekipu prikazuje broj aktivnih članova.

```
SELECT e.Naziv, (SELECT COUNT(*) FROM sport.ClanoviEkipa c WHERE  
c.EkipaID=e.EkipaID AND c.DatumDo IS NULL) AS AktivniClanovi FROM sport.Ekipe e
```

2.12.2. Skupovni operatori

UNION operator kombinuje rezultate dva upita i uklanja duple redove. Primjer prikazuje sve sportiste i sudije u jedinstvenoj listi.

```
SELECT Ime, Prezime, N'Sportista' AS Uloga FROM sport.Sportisti UNION SELECT Ime,  
Prezime, N'Sudija' FROM sport.Sudije
```

UNION ALL operator kombinuje rezultate dva upita i zadržava sve redove, uključujući duple. Primjer prikazuje sve ekipe i organizatore.

```
SELECT Naziv, N'Ekipa' AS Tip FROM sport.Ekipe UNION ALL SELECT Naziv,  
N'Organizator' FROM sport.Organizatori
```

INTERSECT operator vraća redove koji se pojavljuju u oba upita. Primjer prikazuje države koje imaju i sportiste i sudije.

```
SELECT DrzavaID FROM sport.Sportisti INTERSECT SELECT DrzavaID FROM sport.Sudije
```

EXCEPT operator vraća redove iz prvog upita koji se ne pojavljuju u drugom upitu. Primjer prikazuje države koje imaju sportiste, ali nemaju sudije.

```
SELECT DrzavaID FROM sport.Sportisti EXCEPT SELECT DrzavaID FROM sport.Sudije
```

2.13. Korisnici, bezbjednost i uloge

Bezbjednost baze podataka je ključni aspekt svakog informacionog sistema. U SQL Serveru, bezbjednost se ostvaruje kroz korisnike, uloge i dodjelu prava. Uloge omogućavaju grupisanje korisnika sa sličnim potrebama za pristupom podacima.

2.13.1. Kreiranje uloga

Prvi korak u uspostavljanju bezbjednosti jeste kreiranje uloga koje definišu različite nivoe pristupa. U ovom sistemu kreirane su dvije uloge. Uloga db_sportski_operater namijenjena je operativnom osoblju koje radi sa sportskim podacima. Uloga db_sportski_izvjestaji namijenjena je analitičarima koji pripremaju izvještaje.

```
CREATE ROLE db_sportski_operater
```

```
CREATE ROLE db_sportski_izvjestaji
```

2.13.2. Dodjela prava ulogama

Nakon kreiranja uloga, potrebno je dodijeliti im odgovarajuća prava. Operaterima se daje pravo SELECT, INSERT i UPDATE na shemi sport, ali im se uskraćuje pravo DELETE na istoj shemi. Također, operaterima se daje pravo SELECT na shemi sif kako bi mogli vidjeti šifarnike. Analitičarima se daje pravo SELECT na shemi izvjestaji i EXECUTE na procedurama u toj shemi.

```
GRANT SELECT, INSERT, UPDATE ON SCHEMA::sport TO db_sportski_operater
```

```
DENY DELETE ON SCHEMA::sport TO db_sportski_operater
```

```
GRANT SELECT ON SCHEMA::sif TO db_sportski_operater
```

```
GRANT SELECT ON SCHEMA::izvjestaji TO db_sportski_izvjestaji
```

```
GRANT EXECUTE ON SCHEMA::izvjestaji TO db_sportski_izvjestaji
```


2.13.3. Kreiranje korisnika

Korisnici se kreiraju i dodjeljuju odgovarajućim ulogama. U ovom primjeru kreirani su korisnici bez SQL Server login-a, što je pogodno za demonstraciju unutar baze podataka. Korisnik SportskiOperater se dodjeljuje ulozi db_sportski_operater, dok se korisnik SportskiAnaliticar dodjeljuje ulozi db_sportski_izvjestaji.

```
CREATE USER SportskiOperater WITHOUT LOGIN
```

```
CREATE USER SportskiAnaliticar WITHOUT LOGIN
```

```
EXEC sp_addrolemember N'db_sportski_operater', N'SportskiOperater'
```

```
EXEC sp_addrolemember N'db_sportski_izvjestaji', N'SportskiAnaliticar'
```

2.13.4. Testiranje sigurnosnog konteksta

Sigurnosni kontekst se može testirati korištenjem EXECUTE AS USER naredbe, koja mijenja kontekst izvršavanja na određenog korisnika. U primjeru ispod, kontekst se mijenja na analitičara koji zatim čita podatke iz izvještajnog pogleda. Nakon toga, kontekst se vraća na originalnog korisnika pomoću REVERT naredbe.

```
EXECUTE AS USER=N'SportskiAnaliticar'
```

```
SELECT TOP 5 * FROM izvjestaji.vw_RasporedUtakmica
```

```
REVERT
```

Test DELETE zabrane prikazuje da operater ne može izvršiti brisanje podataka iz sheme sport.

2.14. ETL PROCESI

ETL procesi se koriste za ekstrakciju podataka iz izvornih sistema, njihovu transformaciju i učitavanje u ciljnu bazu podataka. U ovom sistemu implementirana su dva ETL procesa.

2.14.1. Tabela StageSportisti

Prvi korak u ETL procesu je kreiranje staging tabele koja prima podatke iz izvornog sistema. Tabela StageSportisti sadrži podatke o sportistima koji trebaju biti uvezeni u glavni sistem. Kolona Obradjen se koristi za praćenje koji redovi su već obrađeni.

U tabelu se unose testni podaci koji predstavljaju sportiste koji čekaju na uvoz.

INSERT INTO

```
etl.StageSportisti(IzvorniID,Ime,Prezime,DatumRodjenja,ISOKod,Email,VisinaCm,TezinaKg)
VALUES
(1001,N'Danilo',N'Jurić','1998-02-10',N'BIH',N'danilo.juric@example.com',184,79),
(1002,N'Petra',N'Novak','2001-07-19',N'HRV',N'petra.novak@example.com',177,65),
(1003,N'Leon',N'Kovač','1999-11-03',N'SVN',N'leon.kovac@example.com',181,76)
```

2.14.2. Procedura sp_UveziSportiste

Prvi ETL proces je implementiran kroz proceduru sp_UveziSportiste. Ova procedura učitava podatke iz staging tabele u glavnu tabelu Sportisti. Prije uvoza, podaci se čiste korištenjem LTRIM i RTRIM funkcija za uklanjanje suvišnih razmaka, a email se pretvara u mala slova. Podaci se povezuju sa šifarnikom država kako bi se dobio odgovarajući identifikator države.

CREATE PROCEDURE etl.sp_UveziSportiste AS

BEGIN

SET NOCOUNT ON; SET XACT_ABORT ON;

BEGIN TRANSACTION;

INSERT INTO

sport.Sportisti(DrzavaID,Ime,Prezime,DatumRodjenja,Email,VisinaCm,TezinaKg)

SELECT

d.DrzavaID,LTRIM(RTRIM(s.Ime)),LTRIM(RTRIM(s.Prezime)),s.DatumRodjenja,LOWER(s.
Email),s.VisinaCm,s.TezinaKg

FROM etl.StageSportisti s INNER JOIN sif.Drzave d ON s.ISOKod=d.ISOKod

```

WHERE s.Obradjen=0 AND NOT EXISTS(SELECT 1 FROM sport.Sportisti x WHERE
x.Email=LOWER(s.Email));
UPDATE etl.StageSportisti SET Obradjen=1 WHERE Obradjen=0;
COMMIT TRANSACTION;
END;

```

Procedura provjerava da li sportista sa istim emailom već postoji u glavnoj tabeli prije uvoza, kako bi se spriječilo dupliranje. Nakon uspješnog uvoza, redovi u staging tabeli se označavaju kao obrađeni. Cijeli proces je obavljen transakcijom.

2.14.3. Tabela DnevniPrihod

Drugi ETL proces zahtijeva kreiranje izvještajne tabele koja će čuvati dnevne prihode po takmičenjima. Tabela DnevniPrihod sadrži datum, identifikator takmičenja, broj ulaznica, ukupan prihod i datum učitavanja.

```

CREATE TABLE izvjestaji.DnevniPrihod(
Datum DATE NOT NULL,TakmicenjeID INT NOT NULL,BrojUlaznica INT NOT
NULL,UkupanPrihod DECIMAL(14,2) NOT NULL,
DatumUcitavanja DATETIME NOT NULL DEFAULT GETDATE(),CONSTRAINT
PK_DnevniPrihod PRIMARY KEY(Datum,TakmicenjeID)
);

```

2.14.4. Procedura sp_UcitajDnevniPrihod

Drugi ETL proces je implementiran kroz proceduru sp_UcitajDnevniPrihod. Ova procedura izračunava dnevni prihod za određeni datum i učitava ga u izvještajnu tabelu.

```

CREATE PROCEDURE etl.sp_UcitajDnevniPrihod @Datum DATE AS
BEGIN
SET NOCOUNT ON; SET XACT_ABORT ON;
BEGIN TRANSACTION;

```

```

DELETE FROM izvjestaji.DnevniPrihod WHERE Datum=@Datum;
INSERT INTO izvjestaji.DnevniPrihod(Datum,TakmicenjeID,BrojUlaznica,UkupanPrihod)
SELECT @Datum,t.TakmicenjeID,COUNT(p.PlacanjeID),ISNULL(SUM(p.Iznos),0)
FROM sport.Takmicenja t
LEFT JOIN sport.Utakmice u ON t.TakmicenjeID=u.TakmicenjeID
LEFT JOIN prodaja.Ulaznice ul ON u.UtakmicaID=ul.UtakmicaID
LEFT JOIN prodaja.Placanja p ON ul.UlaznicaID=p.UlaznicaID AND
p.StatusPlacanja=N'Placeno'
AND p.DatumPlacanja>=@Datum AND p.DatumPlacanja<DATEADD(DAY,1,@Datum)
GROUP BY t.TakmicenjeID;
COMMIT TRANSACTION;
END;

```

Procedura prvo briše sve postojeće zapise za traženi datum, a zatim izračunava broj ulaznica i prihod po takmičenjima za taj datum. Podaci se dovode iz tabela Takmicenja, Utakmice, Ulaznice i Placanja, sa filtriranjem na datum plaćanja i status placeno. Rezultat se učitava u izvještajnu tabelu.

2.15. Kreiranje rezervne kopije baze podataka

Redovno kreiranje rezervnih kopija predstavlja jednu od najvažnijih aktivnosti u administraciji baza podataka. Backup omogućava oporavak sistema u slučaju gubitka podataka usljed greške korisnika, kvara hardvera, softverskih problema ili sigurnosnih incidenata.

2.15.1. Kreiranje potpunog backup-a

Potpuni backup kreira kompletnu kopiju cijele baze podataka, uključujući sve tabele, indekse, procedure i podatke. Ovo je osnovni tip backup-a i predstavlja polaznu tačku za sve ostale vrste backup-a.

U naredbi BACKUP DATABASE navodi se ime baze koju želimo da backup-ujemo, dok se pomoću opcije TO DISK definiše lokacija na kojoj će backup fajl biti sačuvan. Opcije koje se

koriste uključuju INIT za prepisivanje postojećeg sadržaja, CHECKSUM za provjeru integriteta, COMPRESSION za kompresiju backup fajla, NAME za naziv backup seta i STATS za prikaz napretka u procentima.

```
BACKUP DATABASE SportskaTakmicenja TO  
DISK=N'C:\SQLBackup\SportskaTakmicenja_FULL.bak' WITH INIT, CHECKSUM,  
COMPRESSION, NAME=N'Puni backup', STATS=10
```

2.15.2. Diferencijalni backup

Diferencijalni backup čuva samo promjene koje su nastale nakon posljednjeg potpunog backup-a. Ova vrsta backup-a je brža od potpunog i zahtijeva manje prostora, ali se mora kombinovati sa potpunim backup-om za potpunu restauraciju.

```
BACKUP DATABASE SportskaTakmicenja TO  
DISK=N'C:\SQLBackup\SportskaTakmicenja_DIFF.bak' WITH DIFFERENTIAL, INIT,  
CHECKSUM, STATS=10
```

2.15.3. Backup loga transakcija

Ova vrsta backup-a čuva sve transakcije koje su izvršene nad bazom. Omogućava vraćanje baze na tačno određeni trenutak. Da bi se koristio backup loga, baza mora biti u FULL recovery modelu.

```
ALTER DATABASE SportskaTakmicenja SET RECOVERY FULL  
BACKUP DATABASE SportskaTakmicenja TO  
DISK=N'C:\SQLBackup\SportskaTakmicenja_FULL_FOR_LOG.bak' WITH INIT,  
CHECKSUM  
BACKUP LOG SportskaTakmicenja TO  
DISK=N'C:\SQLBackup\SportskaTakmicenja_LOG_001.trn' WITH INIT, CHECKSUM,  
STATS=10
```

2.15.4. Copy-only backup

Copy-only backup se koristi kada želimo napraviti backup bez uticaja na postojeći backup lanac. Ovo je korisno kada treba napraviti dodatnu kopiju bez narušavanja redovnog backup rasporeda.

```
BACKUP DATABASE SportskaTakmicenja TO
```

```
DISK=N'C:\SQLBackup\SportskaTakmicenja_COPY_ONLY.bak' WITH COPY_ONLY, INIT,  
CHECKSUM, COMPRESSION
```

2.15.5. Ostale vrste backup-a

SQL Server podržava i druge vrste backup-a. File backup omogućava backup pojedinačnih datoteka baze. Filegroup backup omogućava backup grupa datoteka. Partial backup kreira kopiju samo primarne filegrupe i čitljivih filegrupa. Striped backup distribuira backup preko više fajlova radi ubrzanja i veće fleksibilnosti. Tail-log backup pravi kopiju posljednjeg dijela loga prije oporavka.

```
BACKUP DATABASE SportskaTakmicenja FILE=N'SportskaTakmicenja' TO
```

```
DISK=N'C:\SQLBackup\SportskaTakmicenja_PRIMARY_FILE.bak' WITH INIT,  
CHECKSUM
```

```
BACKUP DATABASE SportskaTakmicenja FILEGROUP=N'PRIMARY' TO
```

```
DISK=N'C:\SQLBackup\SportskaTakmicenja_PRIMARY_FILEGROUP.bak' WITH INIT,  
CHECKSUM
```

```
BACKUP DATABASE SportskaTakmicenja READ_WRITE_FILEGROUPS TO
```

```
DISK=N'C:\SQLBackup\SportskaTakmicenja_PARTIAL.bak' WITH INIT, CHECKSUM
```

```
BACKUP DATABASE SportskaTakmicenja READ_WRITE_FILEGROUPS TO
```

```
DISK=N'C:\SQLBackup\SportskaTakmicenja_PARTIAL_DIFF.bak' WITH DIFFERENTIAL,  
INIT, CHECKSUM
```

```
BACKUP DATABASE SportskaTakmicenja TO
```

```
DISK=N'C:\SQLBackup\SportskaTakmicenja_STRIPE_1.bak',
```

```
DISK=N'C:\SQLBackup\SportskaTakmicenja_STRIPE_2.bak' WITH INIT, CHECKSUM
```

```
BACKUP DATABASE SportskaTakmicenja TO  
DISK=N'C:\SQLBackup\SportskaTakmicenja_MIRROR_A.bak' MIRROR TO  
DISK=N'D:\SQLBackup\SportskaTakmicenja_MIRROR_B.bak' WITH FORMAT,  
CHECKSUM
```

2.15.6. Informacije o backup-u

Prije restauracije, korisno je dobiti informacije o backup fajlu. HEADERONLY prikazuje zaglavlje backup seta. FILELISTONLY prikazuje listu datoteka unutar backup-a. VERIFYONLY provjerava integritet backup fajla.

```
RESTORE HEADERONLY FROM DISK=N'C:\SQLBackup\SportskaTakmicenja_FULL.bak'  
RESTORE FILELISTONLY FROM DISK=N'C:\SQLBackup\SportskaTakmicenja_FULL.bak'  
RESTORE VERIFYONLY FROM DISK=N'C:\SQLBackup\SportskaTakmicenja_FULL.bak'  
WITH CHECKSUM
```

2.15.7. Restauracija baze iz backup-a

U slučaju gubitka podataka ili potrebe za vraćanjem prethodnog stanja, koristi se naredba RESTORE DATABASE. Opcija REPLACE omogućava prepisivanje postojeće baze, dok MOVE omogućava promjenu lokacije datoteka. STATS prikazuje napredak procesa.

```
RESTORE DATABASE SportskaTakmicenja_TestRestore FROM  
DISK=N'C:\SQLBackup\SportskaTakmicenja_FULL.bak' WITH MOVE N'SportskaTakmicenja'  
TO N'C:\SQLData\SportskaTakmicenja_TestRestore.mdf', MOVE N'SportskaTakmicenja_log'  
TO N'C:\SQLData\SportskaTakmicenja_TestRestore_log.ldf', REPLACE, RECOVERY,  
STATS=10
```

Za kombinaciju full i differential backup-a, prvo se vraća full backup sa NORECOVERY, a zatim differential backup sa RECOVERY. Za point-in-time oporavak, vraća se full backup sa NORECOVERY, a zatim se primjenjuju log backup-ovi sa STOPAT opcijom.

2.16. Publikacija i transakcijska replikacija

Transakcijska replikacija omogućava distribuciju podataka iz jedne baze u drugu, sa minimalnim kašnjenjem. Ovo je korisno za scenarije gdje je potrebno imati kopiju podataka na udaljenoj lokaciji za izvještavanje ili za smanjenje opterećenja primarne baze.

2.16.1. Konfiguracija Distributora

Prvi korak u uspostavljanju replikacije je konfiguracija Distributora. Distributor je server koji čuva podatke o replikaciji i distribuira promjene. U ovom primjeru, Distributor je postavljen na isti server kao i Publisher.

USE master

```
EXEC sp_adddistributor @distributor=@@SERVERNAME,  
@password=N'JakaLozinka_2026!'  
EXEC sp_adddistributiondb @database=N'distribution', @data_folder=N'C:\SQLData',  
@log_folder=N'C:\SQLData'  
EXEC sp_adddistpublisher @publisher=@@SERVERNAME, @distribution_db=N'distribution',  
@security_mode=1, @working_directory=N'\VM1\ReplSnapshot'
```

2.16.2. Kreiranje publikacije

Nakon konfiguracije Distributora, potrebno je kreirati publikaciju koja definiše koje tabele će biti replicirane. Baza podataka se označava kao publikovana, a zatim se kreira publikacija sa određenim svojstvima.

USE SportskaTakmicenja

```
EXEC sp_replicationdboption @dbname=N'SportskaTakmicenja', @optname=N'publish',  
@value=N'true'  
EXEC sp_addpublication @publication=N'SportskaTakmicenja_Pub', @status=N'active',  
@sync_method=N'concurrent', @repl_freq=N'continuous', @description=N'Transakcijska  
publikacija'
```



```
EXEC sp_addpublication_snapshot @publication=N'SportskaTakmicenja_Pub',  
@job_login=N'DOMAIN\sqlrepl', @job_password=N'JakaLozinka_2026!',  
@publisher_security_mode=1
```

2.16.3. Dodavanje artikala

Artikli definišu koje tabele se publikuju. U ovom primjeru, publikuju se tabele Utakmice i Takmicenja iz sheme sport.

```
EXEC sp_addarticle @publication=N'SportskaTakmicenja_Pub', @article=N'Utakmice',  
@source_owner=N'sport', @source_object=N'Utakmice', @type=N'logbased'  
EXEC sp_addarticle @publication=N'SportskaTakmicenja_Pub', @article=N'Takmicenja',  
@source_owner=N'sport', @source_object=N'Takmicenja', @type=N'logbased'
```

2.16.4. Kreiranje pretplate

Pretpatnik je server koji prima replicirane podatke. U ovom primjeru, pretpatnik je VM2, a podaci se repliciraju u bazu SportskaTakmicenja_Sub. Push subscription znači da Publisher gura promjene ka Subscriber-u.

```
EXEC sp_addsubscription @publication=N'SportskaTakmicenja_Pub', @subscriber=N'VM2',  
@destination_db=N'SportskaTakmicenja_Sub', @subscription_type=N'Push',  
@sync_type=N'automatic'  
EXEC sp_addpushsubscription_agent @publication=N'SportskaTakmicenja_Pub',  
@subscriber=N'VM2', @subscriber_db=N'SportskaTakmicenja_Sub',  
@subscriber_security_mode=0, @subscriber_login=N'repl_user',  
@subscriber_password=N'JakaLozinka_2026!'
```

Nakon uspostavljanja replikacije, promjene koje se izvrše na Publisher serveru automatski se prenose na Subscriber server. Ovo se može testirati unošenjem novog reda na Publisher strani i provjerom da li se isti red pojavljuje na Subscriber strani.

2.17. Plan testiranja

Plan testiranja obuhvata provjeru svih funkcionalnosti baze podataka, od kreiranja objekata do izvršavanja složenih upita i administrativnih zadataka.

2.17.1. Test integriteta entiteta

Prvi test provjerava integritet entiteta pokušajem unosa duplog primarnog ključa. Test pokušava da unese državu sa postojećim identifikatorom, što bi trebalo da rezultira greškom.

```
BEGIN TRY
SET IDENTITY_INSERT sif.Drzave ON;
INSERT INTO sif.Drzave(DrzavaID,Naziv,ISOKod) VALUES(1,N'Test drzava',N'TST');
SET IDENTITY_INSERT sif.Drzave OFF;
END TRY
BEGIN CATCH
IF OBJECTPROPERTY(OBJECT_ID(N'sif.Drzave'),N'TableHasIdentity')=1
SET IDENTITY_INSERT sif.Drzave OFF;
SELECT N'Test integriteta entiteta' AS Test, ERROR_MESSAGE() AS OcekivanaGreska;
END CATCH
```

2.17.2. Test domenskog integriteta

Drugi test provjerava domenski integritet pokušajem unosa takmičenja sa negativnim nagradnim fondom. Ovo bi trebalo da bude odbijeno CHECK ograničenjem.

```
BEGIN TRY
INSERT INTO sport.Takmicenja
(SportID,OrganizatorID,SezonaID,Naziv,RangTakmicenja,DatumPocetka,DatumZavrsetka,Statu
sTakmicenja,NagradniFond)
VALUES(1,1,3,N'Neispravno testno takmicenje',N'Lokalno','2026-10-01','2026-10-
02',N'Planirano',-1);
END TRY
```

```
BEGIN CATCH
```

```
SELECT N'Test domenskog integriteta' AS Test, ERROR_MESSAGE() AS OcekivanaGreska;
```

```
END CATCH
```

2.17.3. Test referencijalnog integriteta

Treći test provjerava referencijalni integritet pokušajem unosa ekipe sa nepostojećim sportom. Ovo bi trebalo da bude odbijeno stranim ključem.

```
BEGIN TRY
```

```
INSERT INTO sport.Ekipe(SportID,GradID,Naziv,SkraceniNaziv,DatumOsnivanja,Budzet)
```

```
VALUES(999,1,N'Test ekipa sa pogresnim sportom',N'TESTFK','2020-01-01',1000);
```

```
END TRY
```

```
BEGIN CATCH
```

```
SELECT N'Test referencijalnog integriteta' AS Test, ERROR_MESSAGE() AS
```

```
OcekivanaGreska;
```

```
END CATCH
```

2.17.4. Test triger za dnevnik utakmica

Test provjerava da li trigger za dnevnik utakmica ispravno evidentira promjene. Izvršava se UPDATE nad utakmicom unutar transakcije, a zatim se provjerava da li je zapis upisan u dnevnik.

```
BEGIN TRANSACTION;
```

```
UPDATE sport.Utakmice SET StatusUtakmice=N'U toku' WHERE UtakmicaID=9;
```

```
SELECT TOP 1 * FROM audit.UtakmiceDnevnik WHERE UtakmicaID=9 ORDER BY  
DnevnikID DESC;
```

```
ROLLBACK TRANSACTION
```

2.17.5. Test triger za kapacitet objekta

Test provjerava da li trigger za kapacitet objekta ispravno odbija unos kada broj gledalaca prelazi kapacitet.

```

BEGIN TRY
BEGIN TRANSACTION;
UPDATE sport.Utakmice SET BrojGledalaca=999999 WHERE UtakmicaID=1;
COMMIT TRANSACTION;
END TRY
BEGIN CATCH
IF XACT_STATE() <> 0 ROLLBACK TRANSACTION;
SELECT N'Test trigeri kapaciteta' AS Test, ERROR_MESSAGE() AS OcekivanaGreska;
END CATCH

```

2.17.6. Test procedura

Testovi procedura provjeravaju ispravno izvršavanje sp_EvidentirajRezultat i sp_ProdajUlaznicu. Svaki test je obavljen transakcijom koja se poništava, kako bi početno stanje ostalo nepromijenjeno.

```

BEGIN TRANSACTION;
EXEC sport.sp_EvidentirajRezultat @UtakmicaID=9, @Domaci=2, @Gosti=1,
@BrojGledalaca=2500;
SELECT UtakmicaID, StatusUtakmice, RezultatDomaci, RezultatGosti, BrojGledalaca FROM
sport.Utakmice WHERE UtakmicaID=9;
ROLLBACK TRANSACTION

```

2.17.7. Test ETL procesa

Test ETL procesa provjerava da li se sportisti uspješno uvoze iz staging tabele i da li se redovi označavaju kao obrađeni.

```

SELECT N'Prije ETL-a' AS Faza, * FROM etl.StageSportisti ORDER BY IzvorniID;
EXEC etl.sp_UveziSportiste;

```

```
SELECT N'Poslije ETL-a' AS Faza, SportistaID, Ime, Prezime, Email FROM sport.Sportisti
WHERE Email IN(N'danilo.juric@example.com', N'petra.novak@example.com',
N'leon.kovac@example.com') ORDER BY Email;
SELECT N'Staging nakon ETL-a' AS Faza, * FROM etl.StageSportisti ORDER BY IzvorniID
```

2.17.8. Test sigurnosti

Test sigurnosti provjerava da li korisnici imaju odgovarajuće dozvole. Analitičar treba da može čitati izvještaje, dok operater ne smije izvršiti DELETE operaciju.

```
EXECUTE AS USER=N'SportskiAnaliticar';
SELECT TOP 5 * FROM izvjestaji.vw_RasporedUtakmica;
REVERT
```

```
EXECUTE AS USER=N'SportskiOperater';
DELETE FROM sport.Kazne WHERE KaznaID=-1; -- Ovo bi trebalo da vrati grešku
REVERT
```

2.18.9. Test transakcije

Test transakcije provjerava da li ROLLBACK ispravno poništava promjene. Unosi se privremeni sponzor, broj redova se provjerava prije, tokom i nakon ROLLBACK-a.

```
DECLARE @Prije INT, @Tokom INT, @Poslije INT;
SELECT @Prije=COUNT() FROM prodaja.Sponzori;
BEGIN TRANSACTION;
INSERT INTO prodaja.Sponzori(Naziv,KontaktEmail,Telefon) VALUES(N'Privremeni testni
sponzor', N'test@sponzor.example', N'000111222');
SELECT @Tokom=COUNT() FROM prodaja.Sponzori;
ROLLBACK TRANSACTION;
SELECT @Poslije=COUNT(*) FROM prodaja.Sponzori;
SELECT @Prije AS BrojPrije, @Tokom AS BrojTokom, @Poslije AS BrojPoslijeRollbacka
```

2.18.10. Test indeksa

Test indeksa koristi SET STATISTICS IO i SET STATISTICS TIME za analizu plana izvršavanja upita.

```
SET STATISTICS IO ON;
```

```
SET STATISTICS TIME ON;
```

```
SELECT SportistaID, Ime, Prezime FROM sport.Sportisti WHERE Prezime=N'Pavlovic' OR  
Prezime=N'Pavlović';
```

```
SELECT UtakmicaID, TakmicenjeID, DatumVrijeme, StatusUtakmice FROM sport.Utakmice  
WHERE TakmicenjeID=1 AND StatusUtakmice=N'Zakazana';
```

```
SET STATISTICS IO OFF;
```

```
SET STATISTICS TIME OFF
```

2.19. ER DIJAGRAM I OPIS RELACIJA

ER dijagram predstavlja grafički prikaz strukture baze podataka, odnosno način na koji su podaci organizovani i međusobno povezani. Pomoću ovog dijagrama moguće je lakše razumjeti kako sistem funkcioniše i kako se podaci kreću kroz aplikaciju.

2.19.1. Entiteti i njihovi atributi

Entitet Drzave koristi se za čuvanje podataka o državama. Svaka država ima svoj jedinstveni identifikator, naziv i ISO kod. Ovaj entitet omogućava geografsku kategorizaciju gradova, sportista i sudija. Entitet Gradovi definiše gradove povezane sa državama. Svaki grad ima svoj identifikator, naziv, poštanski broj i broj stanovnika. Gradovi su povezani sa ekipama i sportskim objektima.

Entitet Sportovi predstavlja šifarnik sportskih disciplina. Svaki sport ima svoj naziv, tip, minimalan i maksimalan broj igrača. Sportovi su povezani sa takmičenjima i ekipama. Entitet Organizatori čuva podatke o organizatorima takmičenja. Svaki organizator ima svoj naziv, JIB, email, telefon i datum osnivanja. Entitet Sezone definiše vremenske periode za takmičenja. Svaka sezona ima naziv, datum početka i završetka, te oznaku da li je aktivna.

Entitet Takmicenja predstavlja centralni entitet za sve informacije o takmičenjima. Pored osnovnih podataka kao što su naziv i rang, evidentiraju se i sport, organizator, sezona, datumi, status i nagradni fond. Entitet Faze omogućava podjelu takmičenja na faze, kao što su ligaški dio ili završnica. Svaka faza ima svoj redni broj i datume početka i završetka.

Entitet Ekipe čuva podatke o sportskim ekipama. Svaka ekipa ima svoj naziv, skraćeni naziv, grad, sport, datum osnivanja i budžet. Entitet Sportisti čuva podatke o sportistima. Svaki sportista ima svoje ime, prezime, datum rođenja, pol, email, visinu, težinu i datum registracije. Entitet ClanoviEkipa omogućava praćenje članstva sportista u ekipama kroz vrijeme. Svaki unos ima datum početka i datum završetka članstva.

Entitet Objekti definiše sportske objekte. Svaki objekat ima naziv, adresu, kapacitet, tip i datum otvaranja. Entitet Sudije čuva podatke o sportskim sudijama. Svaka sudija ima svoje ime, prezime, broj licence, datum licence i kategoriju. Entitet Utakmice predstavlja srce takmičarskog dijela sistema. Svaka utakmica povezuje takmičenje, fazu, objekat, domaću i gostujuću ekipu, sudiju, datum i vrijeme, status, rezultat i broj gledalaca. Entitet Nastupi omogućava praćenje individualne statistike sportista na utakmicama. Svaki nastup uključuje broj poena, asistencija i minuta nastupa.

Entitet Kazne evidentira disciplinske mjere izrečene na utakmicama. Svaka kazna ima vrstu, minut i novčani iznos. Entitet Sponzori čuva podatke o sponzorima. Svaki sponzor ima svoj naziv, kontakt email i telefon. Entitet TakmicenjeSponzori omogućava povezivanje takmičenja sa sponzorima, uz evidentiranje iznosa i datuma ugovora. Entitet Ulaznice služi za evidenciju karata za utakmice. Svaka ulaznica ima sektor, red, broj sjedišta, cijenu, datum prodaje i status. Entitet Placanja evidentira finansijske transakcije za ulaznice. Svaka uplata ima datum, iznos, način plaćanja, status i broj transakcije.

2.19.2. Veze između entiteta

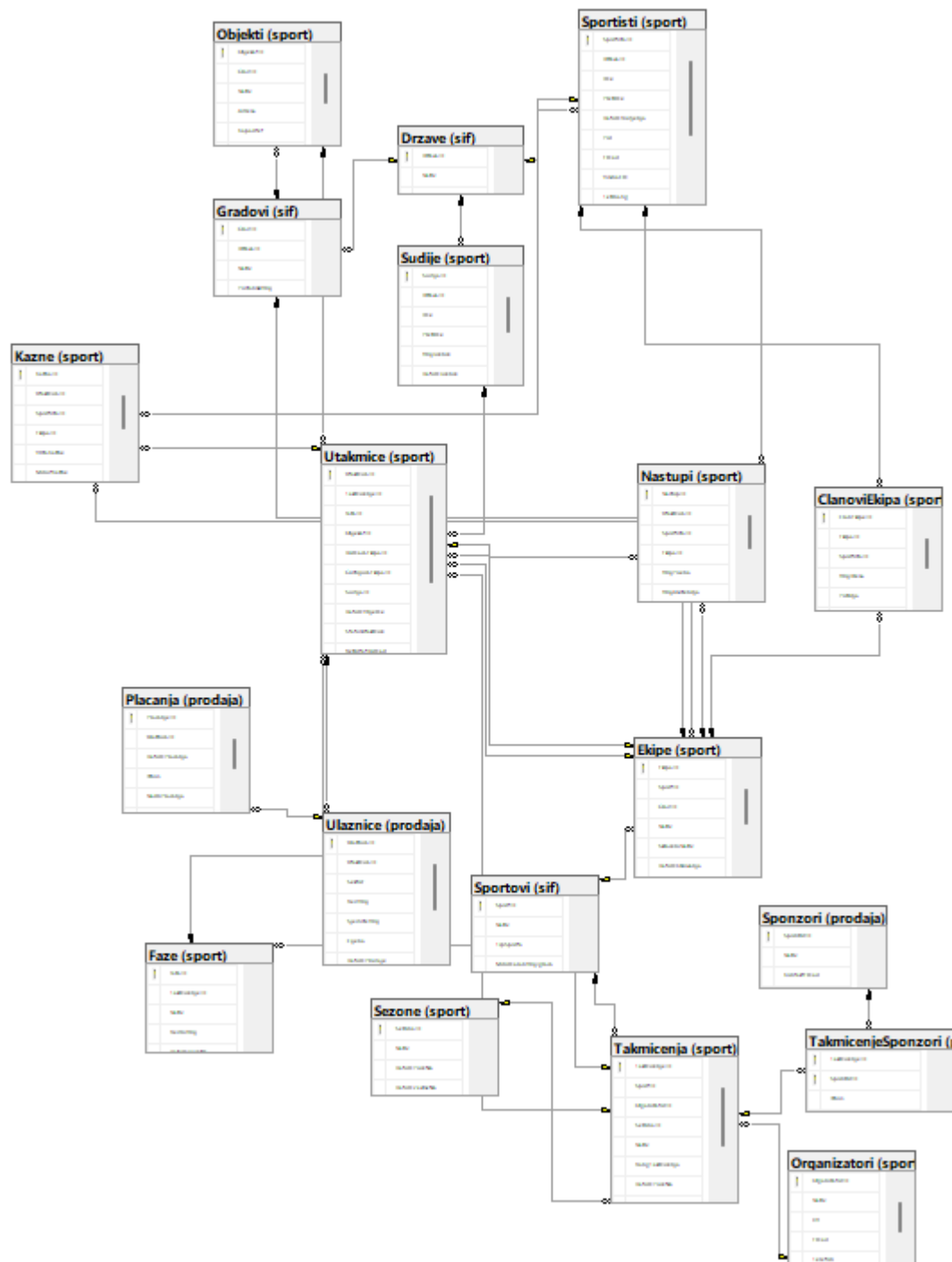
Između entiteta postoje jasno definisane relacije koje omogućavaju povezivanje podataka. Jedna država može imati više gradova, dok svaki grad pripada tačno jednoj državi. Jedna država može imati više sportista i više sudija.

Jedan sport može imati više takmičenja i više ekipa, dok svako takmičenje i svaka ekipa pripadaju tačno jednom sportu. Jedan organizator može organizovati više takmičenja, dok svako takmičenje ima tačno jednog organizatora. Jedna sezona može sadržati više takmičenja, dok svako takmičenje pripada tačno jednoj sezoni.

Jedno takmičenje može imati više faza, a svaka faza pripada tačno jednom takmičenju. Jedno takmičenje može imati više utakmica, dok svaka utakmica pripada tačno jednom takmičenju. Jedna ekipa može imati više sportista kroz članstvo, a jedan sportista može biti član više ekipa tokom vremena. Ova relacija je realizovana kroz tabelu ClanoviEkipa.

Jedan objekat može biti domaćin više utakmica, dok svaka utakmica se održava na tačno jednom objektu. Jedna ekipa može igrati više utakmica kao domaćin ili gost. Jedna utakmica može imati više nastupa sportista, a jedan sportista može imati više nastupa na različitim utakmicama. Jedna utakmica može imati više ulaznica, a jedna ulaznica može imati jedno ili više plaćanja. Jedno takmičenje može imati više sponzora, a jedan sponzor može sponzorirati više takmičenja. Ova relacija je realizovana kroz tabelu TakmicenjeSponzori.

2.19.3. Grafički prikaz ER dijagrama



Slika 22 - ER dijagram baze podataka SportskaTakmicenja

ZAKLJUČAK

U okviru ovog seminarskog rada uspješno je projektovana i implementirana baza podataka SportskaTakmicenja koja pokriva osnovne potrebe sistema za praćenje sportskih takmičenja. Baza se sastoji od dvadeset tri međusobno povezane tabele koje omogućavaju evidenciju država, gradova, sportova, organizatora, sezona, takmičenja, faza, ekipa, sportista, članstava, objekata, sudija, utakmica, nastupa, kazni, sponzora, ulaznica, plaćanja i dnevnika promjena.

Kroz pravilno definisane primarne i strane ključeve obezbijeđen je referencijalni integritet, dok su dodatna ograničenja osigurala tačnost unesenih podataka. Na taj način smanjena je mogućnost grešaka i obezbijeđena pouzdanost sistema.

Korištenjem indeksa unaprijeđene su performanse upita, posebno kod pretrage sportista, utakmica i ulaznica. Takođe, implementirane su procedure, funkcije i trigeri koji automatizuju ključne procese, poput evidentiranja rezultata, prodaje ulaznica i održavanja konzistentnosti podataka.

Dodatno, korišteni su pogledi za jednostavniji prikaz podataka, ETL procesi za uvoz i agregaciju podataka, sigurnosni mehanizmi kroz korisničke uloge, te backup i replikacija za osiguranje dostupnosti i integriteta podataka. Time je formiran cjelovit i funkcionalan model baze podataka koji odgovara savremenim zahtjevima upravljanja sportskim takmičenjima i predstavlja pouzdanu osnovu za dalji razvoj informacionog sistema.

POPIS SLIKA

Slika 1 - Kreiranje i aktiviranje baze podataka	3
Slika 2 - Kreiranje shema u bazi podataka.....	3
Slika 3 - Unos podataka u tabelu Drzave.....	18
Slika 4 - Unos podataka u tabelu Gradovi	19
Slika 5 - Unos podataka u tabelu Sportovi.....	19
Slika 6 - Unos podataka u tabelu Organizatori	19
Slika 7 - Unos podataka u tabelu Sezone.....	20
Slika 8 - Unos podataka u tabelu Takmicenja	20
Slika 9 - Unos podataka u tabelu Faze.....	20
Slika 10 - Unos podataka u tabelu Ekipe	21
Slika 11 - Unos podataka u tabelu Sportisti.....	21
Slika 12 - Unos podataka u tabelu ClanoviEkipa	21
Slika 13 - Unos podataka u tabelu Objekti	22
Slika 14 - Unos podataka u tabelu Sudije	22
Slika 15 - Unos podataka u tabelu Utakmice.....	22
Slika 16 - Unos podataka u tabelu Nastupi	23
Slika 17 - Unos podataka u tabelu Kazne	23
Slika 18 - Unos podataka u tabelu Sponzori	23
Slika 19 - Unos podataka u tabelu TakmicenjeSponzori	24
Slika 20 - Unos podataka u tabelu Ulaznice	24
Slika 21 - Unos podataka u tabelu Placanja.....	24
Slika 22 - ER dijagram baze podataka SportskaTakmicenja	60